HỌC VIỆN CÔNG NGHỆ
BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: LẬP TRÌNH MẠNG

XÂY DỰNG GAME XE TĂNG PvE TANKZORS CHẠY TRÊN NỀN WEB

DEVELOPING TANKZORS: A WEB-BASED PvE TANK GAME

Giảng viên hướng dẫn: TS. Ngô Văn Quyền
Sinh viên thực hiện:
Vũ Anh Tuấn – B23DCCN894
Nguyễn Quang Chiến – B23DCCN094
Trần Quang Lâm – B23DCCN480

Hà Nội, tháng 10 năm 2026

HỌC VIỆN CÔNG NGHỆ
BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: LẬP TRÌNH MẠNG

XÂY DỰNG GAME XE TĂNG PvE TANKZORS CHẠY TRÊN NỀN WEB

DEVELOPING TANKZORS: A WEB-BASED PvE TANK GAME

Giảng viên hướng dẫn: TS. Ngô Văn Quyền
Sinh viên thực hiện:
Vũ Anh Tuấn – B23DCCN894
Nguyễn Quang Chiến – B23DCCN094
Trần Quang Lâm – B23DCCN480

Hà Nội, tháng 10 năm 2026

LỜI CAM ĐOAN

Nhóm thực hiện chịu trách nhiệm về nội dung báo cáo bài tập lớn "Xây dựng game xe tăng PvE Tankzors chạy trên nền web" trong học phần Lập trình mạng. Các tài liệu, khái niệm và công nghệ được sử dụng làm cơ sở cho đề tài được trình bày trong phần tài liệu tham khảo.

Thông tin thành viên, nội dung sản phẩm và hình minh họa được giữ theo báo cáo Tankzors do nhóm cung cấp. Số liệu kiểm thử được ghi rõ mốc lịch sử và phạm vi tư liệu; các nhận định chưa có lần chạy xác nhận mới không được trình bày như kết quả đo mới.

Nhóm đã phân công thực hiện, thảo luận, tổng hợp và rà soát báo cáo. Các thành viên kiểm tra lại nội dung trước khi ký xác nhận và nộp bài.

Hà Nội, tháng 10 năm 2026

Nhóm sinh viên thực hiện

Vũ Anh Tuấn – Nguyễn Quang Chiến – Trần Quang Lâm

LỜI CẢM ƠN

Nhóm chúng em xin chân thành cảm ơn thầy TS. Ngô Văn Quyền, giảng viên học phần Lập trình mạng, đã giảng dạy, định hướng và góp ý trong quá trình học tập cũng như thực hiện bài tập lớn. Những kiến thức về mô hình client – server, giao thức TCP/IP, HTTP và lập trình mạng là nền tảng để nhóm xây dựng hệ thống Tankzors.

Nhóm xin cảm ơn Học viện Công nghệ Bưu chính Viễn thông, Khoa Công nghệ thông tin 1 và các thầy cô đã tạo điều kiện học tập, cung cấp kiến thức chuyên môn để nhóm có thể phân tích, thiết kế và triển khai sản phẩm.

Nhóm cũng cảm ơn các bạn đã trao đổi ý tưởng và góp ý cho trò chơi. Trong quá trình thực hiện, thời gian và kinh nghiệm còn hạn chế nên sản phẩm và báo cáo có thể còn thiếu sót. Nhóm mong nhận được nhận xét của thầy để tiếp tục hoàn thiện.

Nhóm chúng em xin chân thành cảm ơn!

MỤC LỤC

I. GIỚI THIỆU ĐỀ TÀI....1
1.1. Lý do chọn đề tài....1
1.2. Giới thiệu bài toán....1
1.3. Mục tiêu và câu hỏi định hướng....2
1.4. Đối tượng, phạm vi và nguồn tư liệu....2
1.5. Yêu cầu của đề tài....3
1.5.1. Yêu cầu chức năng....3
1.5.2. Yêu cầu phi chức năng....3
1.6. Công nghệ và cơ sở lựa chọn....4
1.6.1. Lý do lựa chọn công nghệ....4
1.7. Phương pháp thực hiện....5
1.8. Bố cục báo cáo....5
II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ....6
2.1. Mô hình client – server....6
2.2. Bộ giao thức TCP/IP và giao thức TCP....6
2.3. Giao thức HTTP và kiến trúc REST....6
2.3.1. Tính idempotent....7
2.4. Mô hình đồng bộ trạng thái trong game mạng....7
2.4.1. Vòng lặp bước cố định và nội suy....8
2.5. Xác thực phiên, BCrypt và CSRF....8
2.6. Các công nghệ nền tảng....8
2.6.1. Spring Boot....8
2.6.2. LibGDX và GWT....8
2.6.3. PostgreSQL, JPA và Flyway....9
2.6.4. Docker....9
2.7. WebSocket và mô hình co-op host-authoritative....9
III. PHƯƠNG PHÁP, PHÂN TÍCH VÀ THIẾT KẾ....10
3.1. Tác nhân và ca sử dụng....10
3.2. Kiến trúc tổng thể....11
3.2.1. Quy tắc phụ thuộc....12
3.2.2. Phân rã module....12
3.3. Thiết kế luồng xử lý....13
3.3.1. Luồng chơi chiến dịch....13
3.3.2. Luồng co-op và giới hạn tích hợp....14
3.4. Thiết kế cơ sở dữ liệu....15

3.5. Thiết kế API....16
3.5.1. REST API....16
3.6. Thiết kế gameplay....17
3.6.1. Bản đồ và nhiệm vụ....17
3.6.2. Thực thể và luật chơi....18
IV. CÀI ĐẶT VÀ KIỂM THỬ HỆ THỐNG....20
4.1. Cấu trúc mã nguồn....20
4.2. Cài đặt phía server....20
4.2.1. Cấp phát màn chơi (bootstrap)....20
4.2.2. Lưu kết quả và chống ghi trùng....21
4.2.3. Tiến trình: mỗi màn chỉ chơi một lần....22
4.2.4. Tiếp tục chiến dịch và bảng xếp hạng theo tài khoản....22
4.2.5. Bảo mật....23
4.3. Cài đặt phía client....23
4.3.1. Vòng lặp mô phỏng bước cố định....23
4.3.2. Di chuyển theo ô và va chạm giữa các xe....24
4.3.3. Trí tuệ nhân tạo của xe tăng....25
4.3.4. Giao tiếp mạng từ trình duyệt....26
4.3.5. Cửa hàng, túi đồ và radar....26
4.3.6. Màn hình kết thúc trận và giao diện tiếng Việt....27
4.4. Giao diện chương trình....27
4.5. Đóng gói và triển khai....31
4.6. Chiến lược kiểm thử....32
4.7. Các ca kiểm thử tiêu biểu....32
4.8. Kết quả kiểm thử và phạm vi xác nhận....34
V. KẾT QUẢ, THẢO LUẬN VÀ KẾT LUẬN....35
5.1. Kết quả đạt được....35
5.2. Hạn chế và giá trị của bằng chứng....35
5.3. Thảo luận theo câu hỏi định hướng....36
5.3.1. Ranh giới mô phỏng và dịch vụ mạng....36
5.3.2. Lưu kết quả và tính bền vững....36
5.3.3. Đóng gói và khả năng kiểm chứng....36
5.4. Kết luận....36
5.5. Hướng phát triển....37
TÀI LIỆU THAM KHẢO....38
PHỤ LỤC....40
Phụ lục A. Cài đặt và sử dụng hệ thống....40
A.1. Yêu cầu môi trường....40

A.2. Khởi động.....40
A.3. Cập nhật, dừng và khởi động lại.....40
Phụ lục B. Thực thi và lưu bằng chứng kiểm thử.....41
Phụ lục C. Phím điều khiển.....42
Phụ lục D. Tóm tắt giao tiếp campaign và co-op.....43
Phụ lục E. Cấu trúc mẫu và đối chiếu từng mục.....44
Phụ lục F. Đối chiếu phiên bản và giới hạn xác nhận.....45

DANH MỤC HÌNH VẼ

Hình 3.1. Sơ đồ ca sử dụng tổng quát.....10
Hình 3.2. Kiến trúc tổng thể của hệ thống Tankzors.....11
Hình 3.3 mô tả trình tự một lượt chơi campaign. Trong các bước cập nhật thế giới, client không cần chờ phản hồi mạng. Bootstrap và lưu kết quả là hai thao tác mạng chính của một lượt chơi; đồng bộ cửa hàng là một thao tác bổ sung khi trạng thái trang bị thay đổi.....13
Hình 3.3. Sơ đồ tuần tự luồng chơi chiến dịch.....14
Hình 3.4. Lược đồ cơ sở dữ liệu.....15
Hình 4.1. Vòng lặp mô phỏng bước cố định.....24
Hình 4.2. Trang đăng nhập.....28
Hình 4.3. Trang chủ sau khi đăng nhập.....28
Hình 4.4. Trang chọn màn: màn đã thắng hiển thị "Đã hoàn thành", chỉ màn kế tiếp được bắt đầu.....29
Hình 4.5. Màn hình nhiệm vụ trước khi vào trận.....29
Hình 4.6. Màn hình trong trận: thanh máu, số đạn, số xe địch còn lại và gợi ý.....30
Hình 4.7. Trang bảng xếp hạng.....30
Hình 4.8. Trang quản trị: thống kê, quản lý tài khoản và bảng xếp hạng.....31
Hình 4.9. Quy trình build Docker nhiều giai đoạn.....31

DANH MỤC BẢNG BIỂU

Bảng 1.1. Công nghệ sử dụng trong dự án.....4
Bảng 2.1. So sánh các mô hình quyền trạng thái trong game mạng.....7
Bảng 3.1. Mô tả các ca sử dụng chính.....10
Bảng 3.2. Các package chính của hệ thống.....12
Bảng 3.3. Mô tả các bảng dữ liệu.....15
Bảng 3.4. Lịch sử migration Flyway.....16
Bảng 3.5. Danh sách endpoint REST chính.....16
Bảng 3.6. Mã trạng thái HTTP được sử dụng.....17
Bảng 3.7. Các màn chơi trong chiến dịch.....18
Bảng 3.8. Nhóm vật phẩm trong cửa hàng.....18
Bảng player_progress lưu các màn người chơi đã mở, kèm cột completed. Danh sách màn được chơi là các màn đã mở và chưa hoàn thành. Khi thắng màn N, PlayerProgressService.markVictory đánh dấu màn N là hoàn thành và mở màn N+1; thua thì được chơi lại chính màn đó. Bootstrap từ chối màn đã hoàn thành bằng mã 403; phía client, màn hình chiến thắng chỉ có hai lựa chọn “tiếp tục” và “về danh sách màn” nên không thể chơi lại màn vừa thắng.....22
Bảng 4.1. Một số ca kiểm thử tiêu biểu.....32
Bảng 4.2. Tổng hợp kết quả kiểm thử tự động.....34
Bảng C.1. Phím điều khiển trong trò chơi.....42
Bảng E.1. Đối chiếu cấu trúc tài liệu mẫu với báo cáo Tankzors.....44
Bảng F.1. Các khác biệt cần kiểm tra theo phiên bản.....45

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
API	Application Programming Interface – giao diện lập trình ứng dụng
AABB	Axis-Aligned Bounding Box – hộp bao thẳng trục dùng kiểm tra va chạm
AI	Artificial Intelligence – trí tuệ nhân tạo (xe tăng do máy điều khiển)
BCrypt	Thuật toán băm mật khẩu có salt và hệ số chi phí
CSRF	Cross-Site Request Forgery – tấn công giả mạo yêu cầu liên trang
DTO	Data Transfer Object – đối tượng truyền dữ liệu
GWT	Google Web Toolkit – trình biên dịch Java sang JavaScript
HTTP	HyperText Transfer Protocol – giao thức truyền siêu văn bản
JPA	Jakarta Persistence API – chuẩn ánh xạ đối tượng – quan hệ
JSON	JavaScript Object Notation – định dạng trao đổi dữ liệu
MVC	Model – View – Controller
PvE	Player versus Environment – người chơi đấu với máy
REST	Representational State Transfer – kiểu kiến trúc API dựa trên HTTP
TCP	Transmission Control Protocol – giao thức điều khiển truyền vận

Host-authoritative: host client quyết định trạng thái trận co-op; guest gửi input và hiển thị snapshot. Relay: thành phần chuyển tiếp thông điệp giữa thành viên trong phòng, không chạy mô phỏng. Snapshot: bản chụp dữ liệu trạng thái phục vụ hiển thị tại một thời điểm.

XÂY DỰNG GAME XE TĂNG PvE TANKZORS CHẠY TRÊN NỀN WEB

I. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trò chơi trực tuyến là một trong những lĩnh vực ứng dụng tiêu biểu của lập trình mạng: một trò chơi phải giải quyết đồng thời các bài toán giao tiếp client – server, đồng bộ trạng thái theo thời gian thực, xác thực người dùng, lưu trữ dữ liệu bền vững và chống gian lận. Vì vậy, xây dựng một trò chơi hoàn chỉnh chạy trên trình duyệt là cách tốt để vận dụng tổng hợp kiến thức của học phần Lập trình mạng vào một sản phẩm cụ thể.

Thể loại bắn xe tăng góc nhìn từ trên xuống có luật chơi đơn giản, dễ tiếp cận nhưng đòi hỏi mô phỏng vật lý, va chạm và trí tuệ nhân tạo đủ tốt. Đề tài "Xây dựng game xe tăng PvE Tankzors chạy trên nền web" được lựa chọn với mong muốn tạo ra một sản phẩm có thể chơi được thật, đồng thời có kiến trúc mạng rõ ràng, dễ chạy và dễ giải thích.

1.2. Giới thiệu bài toán

Tankzors là trò chơi bắn xe tăng PvE (người chơi đấu với máy) chạy trên trình duyệt. Người chơi điều khiển một xe tăng trên bản đồ dạng lưới ô vuông, tiêu diệt các xe tăng và tháp pháo do AI điều khiển để hoàn thành nhiệm vụ của từng màn. Sau mỗi màn thắng, người chơi nhận Sao và Xu: Xu dùng để mua đạn, vật phẩm hỗ trợ và trang bị, Sao dùng để nghiên cứu công nghệ mới trong cửa hàng (Workshop).

Toàn bộ trải nghiệm diễn ra trong trình duyệt: người chơi truy cập website, đăng nhập, chọn màn và chơi ngay trong trang mà không phải cài đặt phần mềm. Kết quả trận, tiến trình và bảng xếp hạng được lưu trên máy chủ nên người chơi có thể tiếp tục trên thiết bị khác chỉ bằng cách đăng nhập lại.

1.3. Mục tiêu và câu hỏi định hướng

Xây dựng website giới thiệu và quản lý trò chơi bằng Spring Boot, Thymeleaf, có đăng ký, đăng nhập và phân quyền.

Xây dựng trò chơi xe tăng bằng LibGDX, biên dịch sang HTML5 để chạy trực tiếp trong trình duyệt mà không cần cài đặt.

Thiết kế REST API cho chế độ chiến dịch (campaign): cấp phát màn, gửi kết quả trận, đồng bộ cửa hàng và tiến trình.

Lưu bền vững tài khoản, kết quả trận, tiến trình mở khoá màn, bảng xếp hạng và cửa hàng vật phẩm trong PostgreSQL.

Đóng gói toàn bộ hệ thống bằng Docker để có thể triển khai bằng một lệnh duy nhất.

Đề tài được triển khai theo ba câu hỏi định hướng: (1) Làm thế nào tổ chức game chạy trong trình duyệt mà vẫn giữ ranh giới rõ giữa mô phỏng và dịch vụ mạng? (2) Làm thế nào lưu kết quả và tiến trình bền vững khi client có thể gửi lại yêu cầu? (3) Làm thế nào đóng gói, kiểm thử và giải thích được hệ thống với các yêu cầu JDK khác nhau? Phần V thảo luận kết quả theo các câu hỏi này.

1.4. Đối tượng, phạm vi và nguồn tư liệu

Đối tượng nghiên cứu là các giao thức và mô hình giao tiếp mạng dùng trong ứng dụng web và trò chơi chạy trên trình duyệt: HTTP/REST, cơ chế phiên (session) dựa trên cookie, khoá idempotency cho yêu cầu lặp lại và các mô hình phân quyền trạng thái game (server-authoritative, client-authoritative).

Phạm vi sản phẩm chính là chiến dịch một người chơi (PvE) gồm ba màn, cửa hàng và nghiên cứu, website quản trị và bảng xếp hạng. Kho mã hiện có nền tảng co-op theo mô hình host-authoritative gồm room, relay và các session host/guest; việc ghép đầy đủ lobby với launcher/game frame chưa được coi là một luồng nhiều người chơi hoàn chỉnh. Triển khai nhiều máy chủ nằm ngoài phạm vi đề tài.

Báo cáo được biên tập từ BaoCao_Tankzors_moi.docx; hình ảnh và một số mô tả chức năng thuộc phiên bản trong tài liệu này. Kho mã đối chiếu có HEAD e4e9599 tại thời điểm biên tập ngày 28/10/2026. Các khác biệt giữa tài liệu và kho mã, cùng giới hạn xác nhận số liệu, được tổng hợp tại Phụ lục F.

1.5. Yêu cầu của đề tài

1.5.1. Yêu cầu chức năng

Người dùng đăng ký, đăng nhập, đăng xuất; mật khẩu được băm an toàn.

Người chơi chọn màn trong danh sách chiến dịch hoặc bấm "Tiếp tục" để vào màn xa nhất đang mở; màn chỉ được mở khi thắng màn trước, và mỗi màn chỉ được chơi một lần (thắng xong thì màn đó bị khoá).

Trò chơi chạy trong trình duyệt với điều khiển bàn phím, có AI địch, AI đồng minh, va chạm, vật phẩm, radar bản đồ và nhiệm vụ; mỗi màn chiến dịch cho người chơi 4 mạng.

Người chơi mua trang bị bằng Xu và nghiên cứu công nghệ bằng Sao trong cửa hàng; trong trận, cửa hàng chỉ mở khi xe đứng trên ô hồi sinh màu xanh lá hoặc sau khi thắng màn.

Kết quả trận được gửi về server, lưu một lần duy nhất, cộng Sao/Xu và cập nhật bảng xếp hạng theo tên tài khoản.

Quản trị viên xem thống kê, khoá/mở khoá và xoá tài khoản.

1.5.2. Yêu cầu phi chức năng

Trò chơi mô phỏng ổn định ở 30 bước/giây, hiển thị mượt trên các màn hình có tần số quét khác nhau.

Toàn bộ chữ trong game (briefing, gợi ý, cửa hàng, túi đồ, màn hình kết quả) hiển thị bằng tiếng Việt có dấu.

Giao tiếp mạng chịu được việc gửi lại yêu cầu (retry) mà không ghi trùng dữ liệu.

Kiến trúc tách bạch: server không chạy mô phỏng gameplay; renderer không sửa trạng thái thế giới.

Triển khai bằng Docker, máy chủ không cần cài sẵn Java, Gradle, Maven hay PostgreSQL.

1.6. Công nghệ và cơ sở lựa chọn

Bảng 1.1. Công nghệ sử dụng trong dự án

Lớp	**Công nghệ**	**Vai trò**
Gameplay, đồ hoạ	Java, LibGDX 1.12.1	Mô phỏng trận đấu, vẽ bản đồ, xe tăng, hiệu ứng
Client trình duyệt	GWT/HTML5, Gradle 8.10.2, JDK 17	Biên dịch mã Java của game sang JavaScript + WebGL
Client desktop	LWJGL3	Chạy thử nhanh trong quá trình phát triển
Web/backend	Spring Boot 3.3.4, Java 21, Maven	Website, REST API
Giao diện web	Thymeleaf, CSS	Render trang phía server
Xác thực	Spring Security, BCrypt	Đăng nhập theo session, phân quyền USER/ADMIN
Lưu trữ	PostgreSQL 16, Spring Data JPA, Flyway	Lưu dữ liệu, quản lý phiên bản schema
Kiểm thử	JUnit 5, Spring Boot Test, MockMvc, H2	Kiểm thử đơn vị và tích hợp
Triển khai	Docker multi-stage, Docker Compose	Build và chạy toàn bộ hệ thống

1.6.1. Lý do lựa chọn công nghệ

Java cho cả client và server. Dùng chung một ngôn ngữ giúp tái sử dụng kiến thức, công cụ kiểm thử và quy ước mã nguồn. LibGDX cho phép viết game một lần bằng Java rồi chạy trên desktop (LWJGL3) để phát triển nhanh và trên trình duyệt (GWT) để phát hành.

Spring Boot cung cấp sẵn Web MVC, Security và JPA trong một hệ sinh thái thống nhất, phù hợp để xây dựng đồng thời website và REST API trên cùng một cổng 8081. PostgreSQL và Flyway đảm bảo dữ liệu bền vững, ràng buộc toàn vẹn (UNIQUE) dùng làm cơ chế chống ghi trùng, và mọi thay đổi schema được ghi lại thành các migration có đánh số.

Docker giải quyết vấn đề khác biệt phiên bản Java: GWT cần JDK 17 trong khi server dùng Java 21. Docker multi-stage cô lập từng bước build nên máy host chỉ cần cài Docker.

1.7. Phương pháp thực hiện

Nghiên cứu tài liệu về giao thức mạng, framework Spring Boot và LibGDX.
Phân tích yêu cầu, thiết kế kiến trúc theo ranh giới trách nhiệm rõ ràng giữa client và server.
Phát triển lặp: mỗi tính năng đi kèm kiểm thử tự động (JUnit 5) và được kiểm tra tích hợp qua Docker.
Mã nguồn được quản lý bằng Git. Quá trình phát triển được tổ chức theo các nhóm chức năng: gameplay, website/API, dữ liệu và triển khai; mỗi thay đổi cần được đối chiếu với ranh giới trách nhiệm giữa client và server.

1.8. Bố cục báo cáo

Theo bố cục của tài liệu mẫu, nội dung gồm năm phần: I – Giới thiệu đề tài; II – Cơ sở lý thuyết và công nghệ; III – Phương pháp, phân tích và thiết kế; IV – Cài đặt và kiểm thử; V – Kết quả, thảo luận và kết luận. Các phần đầu tài liệu gồm lời cam đoan, lời cảm ơn, mục lục và danh mục; phần cuối gồm tài liệu tham khảo và các phụ lục. Bảng đối chiếu từng mục với tài liệu mẫu nằm ở Phụ lục E.

II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Mô hình client – server

Trong mô hình client – server, máy chủ cung cấp dịch vụ và tài nguyên tập trung, còn máy khách gửi yêu cầu và hiển thị kết quả cho người dùng. Ưu điểm của mô hình là dữ liệu được quản lý tập trung, dễ bảo mật và sao lưu; nhược điểm là máy chủ có thể trở thành điểm nghẽn và điểm lỗi duy nhất.

Tankzors áp dụng mô hình này ở cấp ứng dụng web: trình duyệt là client, Spring Boot là server, PostgreSQL là tầng dữ liệu. Riêng với gameplay, hệ thống không đẩy toàn bộ mô phỏng lên server mà phân chia quyền như trình bày ở mục 2.4.

2.2. Bộ giao thức TCP/IP và giao thức TCP

Bộ giao thức TCP/IP gồm bốn tầng: liên kết, Internet (IP), giao vận (TCP/UDP) và ứng dụng (HTTP, DNS…). TCP là giao thức hướng kết nối: trước khi trao đổi dữ liệu, hai bên thực hiện bắt tay ba bước (SYN, SYN-ACK, ACK); trong quá trình truyền, TCP đảm bảo dữ liệu đến đủ, đúng thứ tự và không trùng lặp nhờ số thứ tự, báo nhận và truyền lại.

Trong cách triển khai hiện tại, HTTP của ứng dụng sử dụng kết nối TCP do hạ tầng web cung cấp. TCP xử lý thứ tự dữ liệu và truyền lại; khi mất gói, cơ chế này có thể gây chờ dữ liệu phía sau. Với campaign, mô phỏng chạy cục bộ nên không chờ một vòng trao đổi mạng ở mỗi bước. Mạng vẫn cần khi tải bản đồ, đồng bộ cửa hàng và lưu kết quả; gián đoạn ở các thời điểm này có thể ảnh hưởng việc lưu tiến trình (Fielding et al., 2022).

2.3. Giao thức HTTP và kiến trúc REST

HTTP là giao thức yêu cầu – phản hồi, không trạng thái (stateless): mỗi yêu cầu gồm phương thức (GET, POST, PUT, DELETE…), đường dẫn, phần đầu (header) và thân (body); máy chủ trả về mã trạng thái (200, 403, 404, 409…) cùng dữ liệu. Vì HTTP không lưu trạng thái, ứng dụng web dùng cookie phiên để nhận diện người dùng giữa các yêu cầu.

REST là một kiểu kiến trúc cho hệ thống phân tán; các API của Tankzors vận dụng HTTP và JSON để trao đổi dữ liệu giữa trình duyệt và Spring Boot. Hai thao tác chính của lượt chơi campaign là GET /api/game/bootstrap và POST /api/game/results; ngoài ra POST /api/game/shop/sync đồng bộ cửa hàng và các endpoint tiến trình, bảng xếp hạng phục vụ website (Fielding, 2000).

2.3.1. Tính idempotent

Một thao tác idempotent có hiệu ứng dự kiến khi thực hiện lặp lại giống như khi thực hiện một lần. HTTP định nghĩa GET là phương thức an toàn và idempotent; POST không có bảo đảm idempotent mặc định. Khi client gửi lại kết quả do chưa nhận được phản hồi, server cần nhận diện cùng lượt chơi để tránh cộng thưởng lặp. Tankzors dùng runId cùng dấu vân tay payload cho cơ chế này, trình bày tại mục 4.2.2 (Fielding et al., 2022).

2.4. Mô hình đồng bộ trạng thái trong game mạng

Một vấn đề trung tâm của game mạng là ai nắm quyền quyết định trạng thái trận đấu (authority). Có hai mô hình phổ biến:

Bảng 2.1. So sánh các mô hình quyền trạng thái trong game mạng

Mô hình	Cách hoạt động	Ưu điểm	Nhược điểm
Server-authoritative	Server chạy mô phỏng, client chỉ gửi lệnh và vẽ trạng thái nhận về	Chống gian lận tốt nhất	Server tốn tài nguyên; độ trễ ảnh hưởng trực tiếp tới cảm giác điều khiển
Client-authoritative	Client tự mô phỏng và báo kết quả cho server	Phản hồi tức thì, server nhẹ, chơi được khi mạng chậm	Client có thể gửi kết quả giả; server chỉ kiểm tra giới hạn hợp lệ

Tankzors dùng client-authoritative cho campaign: client chạy mô phỏng và gửi kết quả; server kiểm tra dữ liệu đầu vào, lưu trữ và cập nhật tiến trình. Mô hình này có phản hồi điều khiển nhanh nhưng tạo giới hạn về khả năng xác minh kết quả. Với nền tảng co-op, host client chạy mô phỏng; các guest gửi input và nhận snapshot qua relay Spring tại /coop. Spring không chạy vòng lặp gameplay thứ hai.

2.4.1. Vòng lặp bước cố định và nội suy

Mô phỏng cần một bước thời gian ổn định để tránh phụ thuộc trực tiếp vào tốc độ vẽ khung hình. Bộ tích luỹ cộng thời gian đã trôi qua và thực hiện các bước 1/30 giây; thời gian dư tạo hệ số nội suy để renderer vẽ giữa hai trạng thái. Hệ thống còn giới hạn số bước chạy bù để tránh một khung hình kéo dài làm ứng dụng bị nghẽn (Fiedler, 2004).

2.5. Xác thực phiên, BCrypt và CSRF

Xác thực dựa trên phiên: sau khi đăng nhập thành công, server tạo một phiên và gửi về cookie JSESSIONID; các yêu cầu sau mang cookie này để server nhận diện người dùng. Mật khẩu không bao giờ được lưu dạng rõ mà được băm bằng BCrypt – thuật toán có salt ngẫu nhiên và hệ số chi phí điều chỉnh được, làm cho tấn công dò mật khẩu trở nên tốn kém.

CSRF lợi dụng việc trình duyệt tự gửi cookie để tạo yêu cầu ngoài ý muốn. Cấu hình hiện tại bỏ kiểm tra CSRF cho /api/game/** và /coop; các form Thymeleaf vẫn dùng cơ chế bảo vệ của Spring Security. Cùng origin giúp dùng cookie phiên thuận tiện nhưng không thay thế biện pháp chống CSRF. Khi triển khai rộng, cần rà soát lại việc miễn token, kiểm tra origin và phạm vi dữ liệu client được phép gửi.

2.6. Các công nghệ nền tảng

2.6.1. Spring Boot

Spring Boot là framework Java giúp tạo ứng dụng độc lập với máy chủ web nhúng (Tomcat), cấu hình tự động và các "starter" cho từng chức năng. Dự án dùng các module: Spring Web MVC (controller, REST), Thymeleaf (render HTML phía server), Spring Security (xác thực, phân quyền) và Spring Data JPA (truy cập cơ sở dữ liệu).

2.6.2. LibGDX và GWT

LibGDX là framework phát triển game đa nền tảng bằng Java, cung cấp vòng lặp ứng dụng, đồ hoạ OpenGL/WebGL, âm thanh và xử lý đầu vào. Ở nền tảng web, LibGDX dùng GWT để biên dịch mã Java thành JavaScript. GWT chỉ hỗ trợ một tập con của

thư viện chuẩn Java (ví dụ không có UUID.randomUUID hay đa luồng), nên mã gameplay phải được viết đơn luồng và tránh các API không được mô phỏng.

2.6.3. PostgreSQL, JPA và Flyway

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở. JPA/Hibernate ánh xạ bảng thành lớp Java (entity) và cung cấp repository. Flyway quản lý phiên bản schema bằng các tệp V1__...sql, V2__...sql… được áp dụng tuần tự khi ứng dụng khởi động; Hibernate chạy ở chế độ validate nên chỉ kiểm tra entity khớp schema chứ không tự sửa bảng.

2.6.4. Docker

Docker đóng gói ứng dụng cùng môi trường chạy thành image. Dockerfile nhiều giai đoạn (multi-stage) cho phép dùng image lớn có công cụ build ở các giai đoạn đầu và chỉ giữ lại tệp kết quả trong image chạy cuối cùng. Docker Compose khai báo nhiều container (ứng dụng và cơ sở dữ liệu) cùng mạng, biến môi trường và volume trong một tệp YAML.

2.7. WebSocket và mô hình co-op host-authoritative

HTTP phù hợp với trao đổi dữ liệu theo yêu cầu, còn co-op cần kênh hai chiều tồn tại trong thời gian phòng hoạt động. WebSocket /coop mang JSON envelope v2. Host chạy GameSimulation và phát WORLD_STATE; guest gửi PLAYER_INPUT và hiển thị snapshot nhận về. Spring kiểm tra membership, quyền host và relay gói tin; không xử lý AI, va chạm hay vòng lặp thế giới.

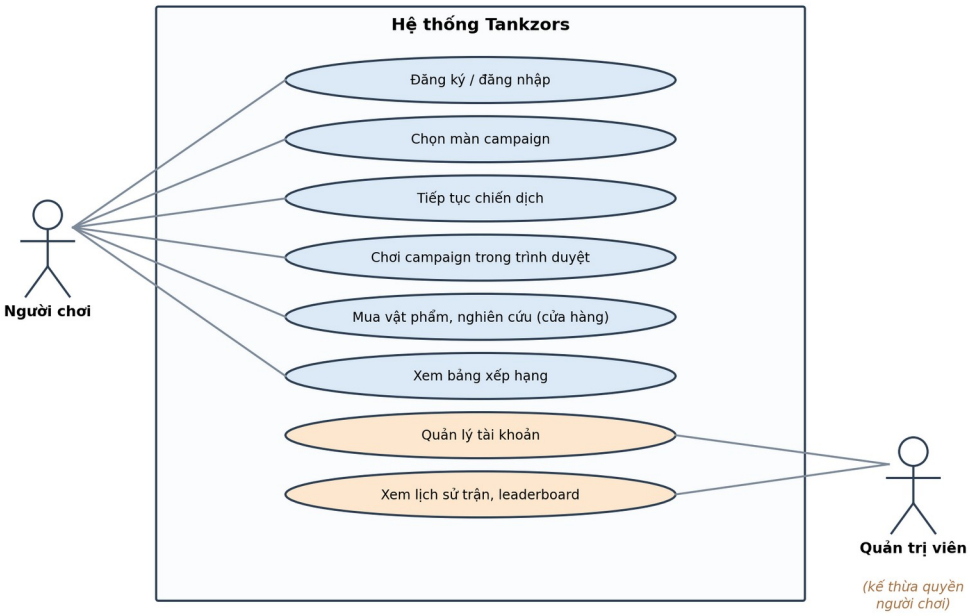
Mô hình host-authoritative tiết kiệm tài nguyên server nhưng phụ thuộc chất lượng kết nối và độ tin cậy của host. Khi host rời phòng, cách xử lý hiện tại đóng phòng; chuyển quyền host và phục hồi trận là hướng phát triển, không được mặc định coi là tính năng đã có.

III. PHƯƠNG PHÁP, PHÂN TÍCH VÀ THIẾT KẾ

Đề tài áp dụng quy trình phát triển phần mềm theo vòng lặp: xác định yêu cầu, phân tích ca sử dụng, thiết kế ranh giới và dữ liệu, cài đặt từng chức năng, kiểm thử rồi điều chỉnh. Phương pháp đánh giá là kiểm thử phần mềm và đối chiếu kiến trúc; báo cáo không sử dụng khảo sát người học hay số liệu đánh giá trải nghiệm chưa được thu thập.

3.1. Tác nhân và ca sử dụng

Hệ thống có hai tác nhân: Người chơi (ROLE_USER) và Quản trị viên (ROLE_ADMIN). Quản trị viên có mọi quyền của người chơi và thêm quyền truy cập trang quản trị.



Hình 3.1. Sơ đồ ca sử dụng tổng quát

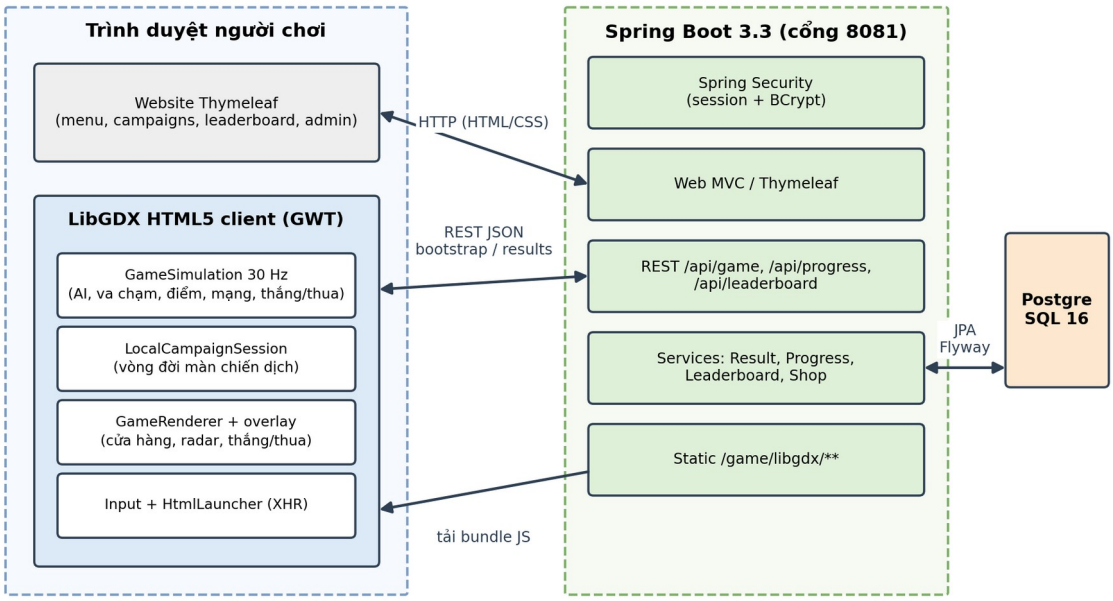
Bảng 3.1. Mô tả các ca sử dụng chính

Ca sử dụng	Tác nhân	Mô tả
Đăng ký / đăng nhập	Người chơi	Tạo tài khoản, đăng nhập bằng form; mật khẩu băm BCrypt
Chọn màn campaign	Người chơi	Xem danh sách 3 màn; màn đang mở có nút Bắt đầu, màn đã thắng hiện “Đã hoàn thành”

Ca sử dụng	Tác nhân	Mô tả
Chơi campaign	Người chơi	Tải bản đồ, chơi trong trình duyệt, kết quả gửi về server khi thắng/thua
Tiếp tục chiến dịch	Người chơi	Nút Tiếp tục ở Game Menu mở màn xa nhất đang được chơi; thắng hết thì báo đã hoàn thành chiến dịch
Mua vật phẩm	Người chơi	Dùng Xu mua đạn, vật phẩm, trang bị; dùng Sao nghiên cứu công nghệ. Trong trận chỉ mở khi đứng trên ô hồi sinh xanh lá
Xem bảng xếp hạng	Người chơi	Xem 10 điểm cao nhất của mỗi màn; mỗi tài khoản chỉ xuất hiện một lần với tên đăng nhập
Quản lý tài khoản	Quản trị viên	Xem thống kê, khoá/mở khoá, xoá tài khoản người chơi

3.2. Kiến trúc tổng thể

Hệ thống được chia thành ba khối với ranh giới trách nhiệm rõ ràng như Hình 3.2.



Hình 3.2. Kiến trúc tổng thể của hệ thống Tankzors

LibGDX client là nguồn sự thật của trạng thái trận: GameSimulation chạy bước cố định, cập nhật xe tăng, AI, đạn, va chạm, điểm và trạng thái thắng/thua. GameRenderer chỉ vẽ snapshot; ClientInputProcessor chỉ phát lệnh vào phiên chơi.

Spring Boot không có thực thể xe tăng, AI hay vòng lặp game. Server chỉ xác thực, render website, cấp bản đồ và runId, kiểm tra và lưu kết quả, quản lý tiến trình, bảng xếp hạng và quản trị.

PostgreSQL lưu tài khoản, kết quả trận, tiến trình, tổng sao/xu, cửa hàng và bảng xếp hạng; schema do Flyway quản lý.

Client HTML5 sau khi build là các tệp tĩnh được đóng gói bên trong app.jar và được chính Spring Boot phục vụ tại /game/libgdx/**, nên client và server dùng chung một cổng và một nguồn gốc (origin). Nhờ vậy cookie phiên tự động đi kèm mọi lời gọi REST mà không cần cấu hình CORS.

3.2.1. Quy tắc phụ thuộc

Gameplay mới chỉ được thêm trong game-client/core; server không phụ thuộc lớp gameplay của client.

Renderer nhận snapshot và không sửa thế giới.

Campaign giao tiếp với server qua REST, không mở WebSocket. Nền tảng co-op dùng WebSocket /coop với JSON envelope v2; namespace /game/libgdx/** chỉ phục vụ tài nguyên HTML5.

REST DTO và entity lưu trữ tách biệt khỏi entity gameplay.

3.2.2. Phân rã module

Bảng 3.2. Các package chính của hệ thống

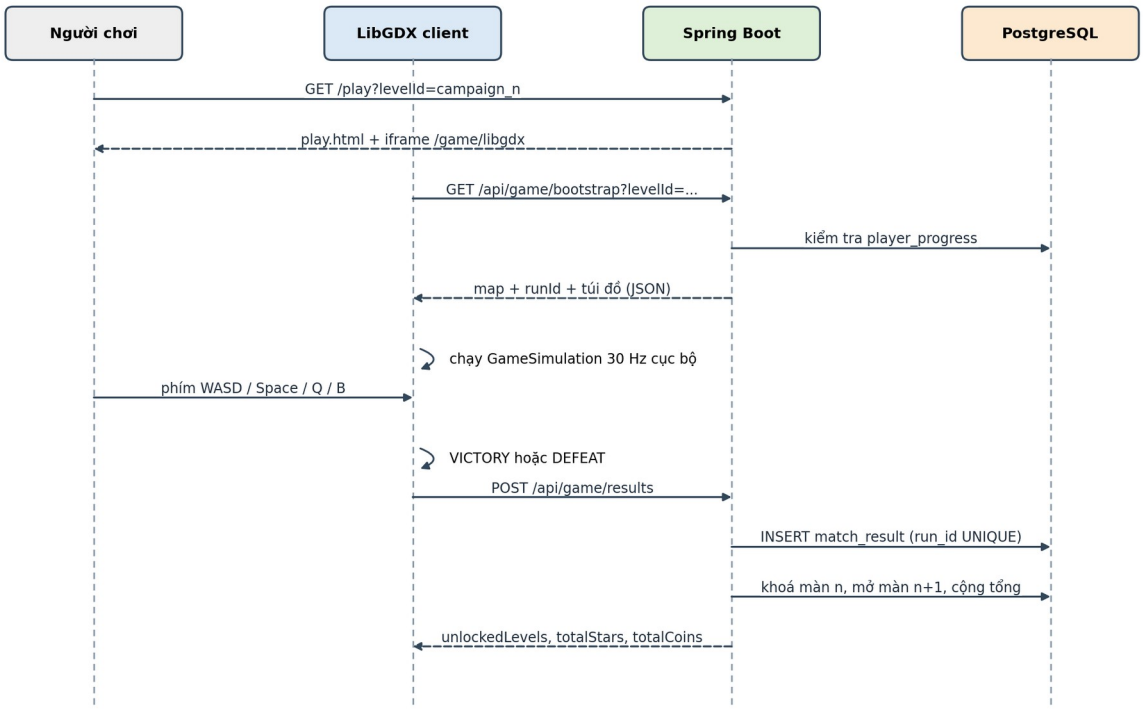
Package	Trách nhiệm
client/gameplay	Mô phỏng: GameSimulation, GameWorld, entity, collision, map, rules
client/gameplay/ai	AI xe tăng (BuildsBotBrain), bắn của tháp pháo, độ khó
client/session	Vòng đời phiên chơi chiến dịch: LocalCampaignSession
client/render, client/input	Vẽ snapshot (GameRenderer), radar bản đồ (RadarOverlay); bàn phím và menu túi đồ (ClientInputProcessor)
client/ui, client/shop	Cửa hàng modal (ShopOverlay, ShopCatalog), màn hình thắng/thua (VictoryOverlay, DefeatOverlay); túi đồ và cây nghiên cứu (PlayerShopState)

Package	Trách nhiệm
client/html	HtmlLauncher, cầu nối XMLHttpRequest của trình duyệt
server/web	Controller trang web và REST API
server/application	Use case: tiến trình, bảng xếp hạng, cửa hàng, truy vấn màn
server/result	Lưu và chống trùng kết quả trận
server/progress, leaderboard	Entity và repository JPA
server/security, identity	Đăng nhập, phân quyền, quản trị; xác định danh tính người chơi, tách tên tài khoản (PlayerNames)
server/map	DTO và bộ nạp bản đồ JSON phục vụ API

3.3. Thiết kế luồng xử lý

3.3.1. Luồng chơi chiến dịch

Hình 3.3 mô tả trình tự một lượt chơi campaign. Trong các bước cập nhật thế giới, client không cần chờ phản hồi mạng. Bootstrap và lưu kết quả là hai thao tác mạng chính của một lượt chơi; đồng bộ cửa hàng là một thao tác bổ sung khi trạng thái trang bị thay đổi.



Hình 3.3. Sơ đồ tuần tự luồng chơi chiến dịch

Người chơi mở /play?levelId=campaign_n, hoặc /play không kèm tham số khi bấm Tiếp tục (server tự chọn màn xa nhất đang mở). Server kiểm tra màn còn được chơi rồi trả trang play.html có iframe nhúng client LibGDX; nếu không, chuyển về /campaigns kèm thông báo.

Client gọi GET /api/game/bootstrap. Server kiểm tra màn đã được mở và chưa hoàn thành, sinh runId ngẫu nhiên, trả về bản đồ, nhiệm vụ, phần thưởng và túi đồ.

Client chạy GameSimulation ở 30 Hz cho tới khi thắng (VICTORY) hoặc thua (DEFEAT).

Client gửi POST /api/game/results kèm runId. Server lưu kết quả, kẹp phần thưởng theo bản đồ; nếu thắng thì khoá màn vừa chơi, mở màn kế tiếp, cộng tổng Sao/Xu và cập nhật bảng xếp hạng. Client hiển thị báo cáo nhiệm vụ, người chơi vào cửa hàng rồi sang màn kế.

3.3.2. Luồng co-op và giới hạn tích hợp

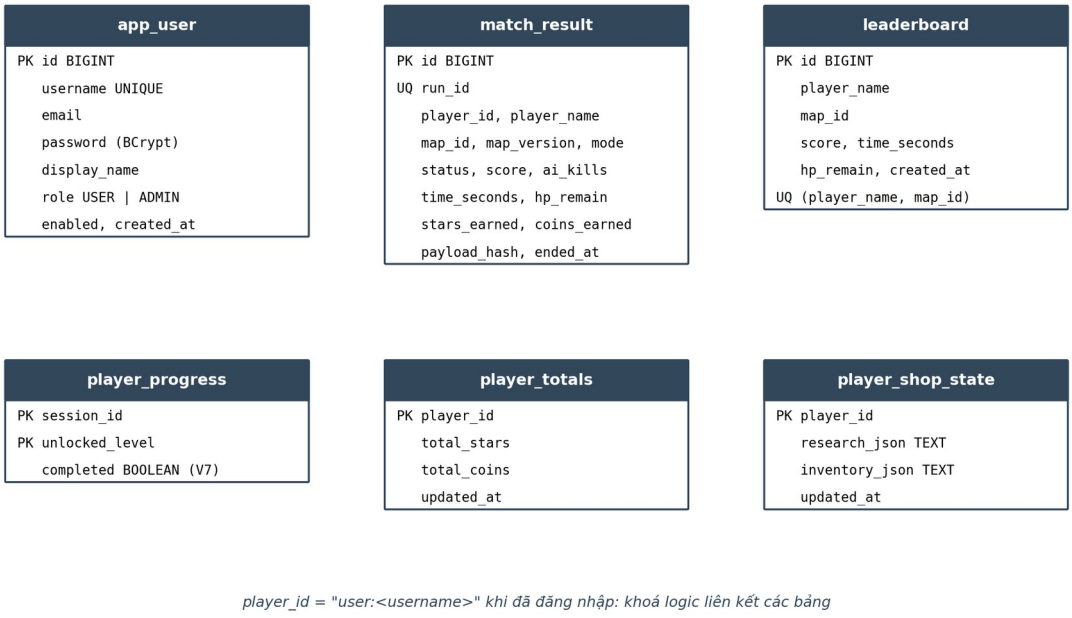
RoomRegistry quản lý thành viên và trạng thái phòng. Kết nối được nhận diện từ handshake, sau đó CREATE_ROOM/JOIN_ROOM/READY/START_MATCH điều phối phòng; PLAYER_INPUT đi từ guest tới host, còn WORLD_STATE và

MATCH_RESULT đi từ host tới guest. Envelope có version, type và có thể có roomId, runId, seq, payload. Relay gắn senderId từ kết nối, không tin danh tính do client tự khai.

HostGameSession và GuestGameSession thể hiện sự phân chia authority trong client. Tài liệu kiến trúc của kho mã xác định phần nối lobby với launcher/game frame vẫn cần hoàn thiện; vì vậy kết quả campaign và nền tảng co-op được đánh giá riêng. Không sử dụng protocol v1 hoặc WebSocket /game cho luồng mục tiêu này.

3.4. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu gồm sáu bảng (Hình 3.4). Các bảng liên kết với nhau qua khoá logic player_id; với người dùng đã đăng nhập, giá trị này có dạng user:<username> nên dữ liệu không đổi khi cookie phiên được tạo lại.



Hình 3.4. Lược đồ cơ sở dữ liệu

Bảng 3.3. Mô tả các bảng dữ liệu

Bảng	Khoá	Nội dung
app_user	id; username UNIQUE	Tài khoản, mật khẩu BCrypt, vai trò, trạng thái kích hoạt

Bảng	Khoá	Nội dung
match_result	id; run_id UNIQUE	Lịch sử trận: màn, chế độ, kết quả, điểm, số xe hạ, thời gian, sao/xu nhận, payload_hash
leaderboard	id; (player_name, map_id) UNIQUE	Điểm cao nhất của mỗi người chơi trên mỗi màn
player_progress	(session_id, unlocked_level)	Màn đã mở; cột completed = TRUE nghĩa là đã thắng và bị khoá
player_totals	player_id	Tổng sao và xu tích luỹ
player_shop_state	player_id	Cây nghiên cứu và túi đồ dạng JSON

Bảng 3.4. Lịch sử migration Flyway

Phiên bản	Nội dung thay đổi
V1__init	Tạo bảng leaderboard, match_result
V2__player_progress	Lưu tiến trình mở khoá màn theo người chơi
V3__match_result_dedup	Thêm run_id UNIQUE, payload_hash, ai_kills, player_id để chống ghi trùng
V4__player_totals	Bảng tổng sao/xu; cột stars_earned, coins_earned
V5__app_users	Bảng tài khoản phục vụ đăng nhập và phân quyền
V6__player_shop_state	Lưu cửa hàng, túi đồ, cây nghiên cứu
V7__player_progress_completed	Cột completed: mỗi màn chỉ chơi một lần, thắng xong bị khoá

3.5. Thiết kế API

3.5.1. REST API

Bảng 3.5. Danh sách endpoint REST chính

Phương thức	Đường dẫn	Chức năng
GET	/api/game/bootstrap?levelId=	Kiểm tra quyền vào màn, trả bản đồ, nhiệm vụ, runId, túi đồ

Phương thức	Đường dẫn	Chức năng
POST	/api/game/results	Lưu kết quả trận (idempotent theo runId), mở khoá màn kế
POST	/api/game/shop/sync	Đồng bộ túi đồ, cây nghiên cứu, số xu
GET	/api/progress	Danh sách màn đang được chơi
POST	/api/progress/reset	Đặt lại tiến trình về màn 1
GET	/api/leaderboard/{mapId}	Bảng xếp hạng một màn
GET	/api/health	Kiểm tra tình trạng ứng dụng

Bảng 3.6. Mã trạng thái HTTP được sử dụng

Mã	Ý nghĩa trong hệ thống
200 OK	Thành công; kể cả khi gửi lại đúng kết quả đã lưu (alreadySaved = true)
403 Forbidden	Bootstrap một màn chưa mở hoặc đã hoàn thành
404 Not Found	Không tìm thấy bản đồ
409 Conflict	Cùng runId nhưng nội dung kết quả khác lần gửi trước

Ví dụ thân yêu cầu gửi kết quả trận:

```
POST /api/game/results
Content-Type: application/json
{
  "runId": "run-3f6c...", "levelId": "campaign_2", "mapVersion": "v1",
  "mode": "CAMPAIGN", "status": "VICTORY", "score": 2110,
  "hpRemain": 5, "aiKills": 6, "elapsedSeconds": 84,
  "starsEarned": 3, "coinsEarned": 200
}
```

3.6. Thiết kế gameplay

3.6.1. Bản đồ và nhiệm vụ

Bản đồ là lưới ô vuông 40 × 40 pixel lưu dưới dạng JSON, gồm lưới va chạm, lưới hình ảnh, vị trí sinh, nhiệm vụ và phần thưởng. Cùng một tệp JSON được đặt ở tài nguyên client và tài nguyên Spring để server có thể kiểm tra phần thưởng tối đa.

Bảng 3.7. Các màn chơi trong chiến dịch

Màn	Kích thước	Độ khó	Nhiệm vụ	Thưởng tối đa
Màn 1 – Cướp cờ	16 × 22	Dễ	Làm quen điều khiển, cướp cờ đối phương	3 sao, 100 xu
Màn 2 – Tường thành	19 × 19	Trung bình	Cùng xe đồng minh tiêu diệt 6 xe địch sinh từ 2 căn cứ	3 sao, 200 xu
Màn 3 – Sa mạc	16 × 22	Khó	Tiêu diệt 7 xe địch và tháp pháo; cát làm chậm xe	3 sao, 300 xu

3.6.2. Thực thể và luật chơi

Xe người chơi: 40 × 40 pixel, tốc độ 60 pixel/giây, 5 máu, thời gian hồi bắn 0,5 giây; di chuyển theo từng ô bằng WASD hoặc phím mũi tên, X để lùi, Space hoặc J để bắn.
Mạng: mỗi màn chiến dịch có 4 mạng (tính cả xe đang điều khiển). Bị phá huỷ thì mất một mạng, mất giáp tạm thời và khiên đang lắp rồi hồi sinh tại ô xuất phát; túi đồ còn lại được giữ nguyên. Hết 4 mạng mà còn vật phẩm Mạng bổ sung thì vật phẩm được dùng tự động; hết hẳn thì thua.
Xe AI: chỉ số máu, tốc độ, loại đạn theo loại xe (0..7) và cấp độ (1..5); có phe địch và phe đồng minh.
Vật cản: tường gạch (phá được), tường đá, bụi cây (che tầm nhìn), cát (làm chậm), nước.
Điểm khi thắng: điểm = máu còn lại × 10 + số xe hạ × 200 + max(0, 1000 − thời gian × 10).
Va chạm: các xe không được đi xuyên nhau; một ô chỉ có một xe đứng hoặc giữ chỗ.

Bảng 3.8. Nhóm vật phẩm trong cửa hàng

Nhóm	Ví dụ	Tác dụng
Đạn cơ bản	Đạn thường, Plasma, Đạn đôi, Plasma đôi, Đạn xuyên giáp, Xuyên giáp cường hóa	Mua theo lốc 10 viên; tăng sát thương, bắn đạn đôi, xuyên giáp

Nhóm	Ví dụ	Tác dụng
Vũ khí đặc biệt	Pháo cầu vồng, Đầu đạn tầm xa, Tên lửa dẫn đường, Mìn TMD-5/TMD-9, Thuốc nổ x70/x100/x160, Không kích	Bay qua tường, nổ vùng, phá vật cản; Không kích cần có Radar
Xe đồng minh	Xe đồng minh cấp 1, cấp 2, cấp 3	Gọi xe tăng đồng minh yểm trợ
Phòng thủ	Giáp tạm thời, Giáp vĩnh cửu, Khiên năng lượng	+1 HP tạm thời hoặc vĩnh viễn; khiên tự tái tạo
Sửa chữa	Bộ sửa xe; Sửa xe của bạn, Sửa xe đồng minh	Hồi HP trong trận; sửa tại cửa hàng 6 Xu/HP (3 Xu/HP khi nghiên cứu giáp cấp 3)
Hỗ trợ	Radar, Khung gầm mở rộng, Mạng bổ sung, Tăng tốc Nitro, Đóng băng	Mở bản đồ tổng quan, tăng sức chứa đạn lên 99, thêm lượt hồi sinh, tăng tốc, đóng băng địch

Ngoài vật phẩm mua bằng Xu, cửa hàng có 10 nhánh nghiên cứu trả bằng Sao: đạn pháo, pháo binh, tên lửa, mìn, thuốc nổ, giáp trụ, khiên lực, không kích, xe đồng minh và Thưởng Xu bổ sung (+5% / +10% Xu sau mỗi chiến thắng). Vật phẩm nâng cao chỉ mua được khi nhánh nghiên cứu tương ứng đạt cấp yêu cầu, ví dụ Plasma cần nghiên cứu đạn pháo cấp 1, Tên lửa dẫn đường cần nghiên cứu tên lửa cấp 2.

IV. CÀI ĐẶT VÀ KIỂM THỬ HỆ THỐNG

4.1. Cấu trúc mã nguồn

Cấu trúc thư mục dự án

```
LapTrinhMang/
├── game-client/        LibGDX: core, html (GWT), desktop, assets, tools
│   └── core/src/main/java/com/tankzors/client/
│       ├── gameplay/   GameSimulation, GameWorld, ai/, collision/, entity/, map/
│       ├── session/    LocalCampaignSession, GameSession
│       ├── render/     GameRenderer, RadarOverlay, VisibilityMask, TrackMarks
│       ├── ui/         ShopOverlay, ShopCatalog, VictoryOverlay, DefeatOverlay
│       └── input/, shop/, assets/
├── server-spring/      Spring Boot: web, application, result, progress,
security,
│   │                   identity, leaderboard, map
│   └── src/main/resources/  templates/, static/, maps/, db/migration/
├── infra/              docker-compose.yml (app + PostgreSQL)
└── docs/               kiến trúc, networking, protocol, runbook
```

4.2. Cài đặt phía server

4.2.1. Cấp phát màn chơi (bootstrap)

Khi nhận yêu cầu bootstrap, controller GameApiController xác định người chơi qua PlayerIdentityResolver, kiểm tra quyền vào màn rồi mới đọc bản đồ. Mỗi lần bootstrap sinh một runId mới bằng UUID.randomUUID(); giá trị này là khoá idempotency cho kết quả của đúng lượt chơi đó.

Trích GameApiController.bootstrap

```
String playerId = identity.resolve(request);
if (!progress.isUnlocked(playerId, levelId)) {
    return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(Map.of("error", "Level chưa unlock: " + levelId));
}
GameMap map = MapLoader.load(levelId);
String runId = "run-" + UUID.randomUUID();
```

Danh tính người chơi được xác định theo thứ tự ưu tiên: tài khoản đã đăng nhập (user:<tên>), header X-Player-Session, thuộc tính trong HttpSession, cuối cùng là tạo phiên mới. Cùng một bộ giải quyết danh tính được dùng cho trang web và REST API nên tiến trình luôn nhất quán.

4.2.2. Lưu kết quả và chống ghi trùng

ResultService.submit xử lý kết quả theo ba nhánh dựa trên run_id (ràng buộc UNIQUE) và payload_hash (băm FNV-1a của các trường kết quả):

runId chưa tồn tại: chèn bản ghi mới, kẹp phần thưởng theo bản đồ, cộng tổng và mở khoá màn nếu thắng.

runId đã tồn tại và cùng hash: đây là yêu cầu gửi lại, trả về kết quả đã lưu với alreadySaved = true, không cộng thưởng lần nữa.

runId đã tồn tại nhưng khác hash: ném ConflictException, trả mã 409.

Trích ResultService.submit

```
MatchResultEntity existing = resultRepo.findByRunId(payload.runId).orElse(null);
if (existing != null) {
    if (hash.equals(existing.getPayloadHash())) {
        return SubmitOutcome.idempotent(view(existing, playerId)); // retry hợp
lệ
    }
    throw new ConflictException("runId " + payload.runId + " đã tồn tại với
payload khác");
}
...
int[] reward = clampReward(payload, row);   // DEFEAT -> 0; VICTORY <=
map.rewards
```

Kết quả do client báo cáo chỉ được kiểm tra về cấu trúc và một số giới hạn giá trị. Server kẹp sao/xu theo phần thưởng tối đa của bản đồ; trận thua hoặc mode không phải campaign không nhận phần thưởng campaign. Score chỉ được kiểm tra khoảng hợp lệ, không được tính lại từ một mô phỏng server. Cơ chế này giúp hạn chế dữ liệu bất thường nhưng chưa chứng minh trận đã diễn ra hợp lệ.

4.2.3. Tiến trình: mỗi màn chỉ chơi một lần

Bảng player_progress lưu các màn người chơi đã mở, kèm cột completed. Danh sách màn được chơi là các màn đã mở và chưa hoàn thành. Khi thắng màn N, PlayerProgressService.markVictory đánh dấu màn N là hoàn thành và mở màn N+1; thua thì được chơi lại chính màn đó. Bootstrap từ chối màn đã hoàn thành bằng mã 403; phía client, màn hình chiến thắng chỉ có hai lựa chọn "tiếp tục" và "về danh sách màn" nên không thể chơi lại màn vừa thắng.

Trích PlayerProgressService.markVictory

```
PlayerProgress won = rows.get(levelId);
if (won == null) won = new PlayerProgress(sessionId, levelId);
if (!won.isCompleted()) {
    won.setCompleted(true);            // khoá màn vừa thắng
    repository.save(won);
}
String nextLevel = CAMPAIGN_ORDER.get(idx + 1);
if (!rows.containsKey(nextLevel)) {
    repository.save(new PlayerProgress(sessionId, nextLevel));   // mở màn kế
tiếp
    return nextLevel;
}
```

4.2.4. Tiếp tục chiến dịch và bảng xếp hạng theo tài khoản

Trang Game Menu có nút Tiếp tục. PageController gọi PlayerProgressService.getContinueLevel để lấy màn xa nhất theo thứ tự chiến dịch trong số các màn đang được chơi (đã mở và chưa hoàn thành); nếu không còn màn nào, nút bị vô hiệu hoá kèm dòng "Bạn đã hoàn thành toàn bộ chiến dịch". Khi bấm Tiếp tục, trình duyệt mở /play không kèm levelId và server tính lại màn cần chơi ngay lúc đó, vì trang menu có thể đã cũ nếu người chơi vừa thắng một màn ở tab khác.

Trích PlayerProgressService.getContinueLevel

```
public synchronized Optional<String> getContinueLevel(String sessionId) {
    List<String> unlocked = getUnlocked(sessionId);  // đã mở, chưa hoàn thành
    return CAMPAIGN_ORDER.stream()
        .filter(unlocked::contains)
        .reduce((previous, current) -> current);   // màn xa nhất
}
```

Trang /play cũng kiểm tra isUnlocked trước khi trả về trang game, nên việc gõ trực tiếp đường dẫn tới một màn đã thắng hoặc chưa mở sẽ bị chuyển về /campaigns kèm thông báo, thay vì chỉ bị chặn ở bước bootstrap. Nếu không đọc được tiến trình (ví dụ

cơ sở dữ liệu tạm thời lỗi), server chuyển về /game-menu với thông báo "Chưa tải được tiến độ" chứ không tự đoán là màn 1.

Với bảng xếp hạng, ResultService ghi tên hiển thị bằng tên tài khoản (phần sau tiền tố user: do PlayerNames.accountName tách ra); người chơi khách vẫn dùng khoá dạng Player-xxxxxx. Khi đọc, LeaderboardService.topScores duyệt điểm theo thứ tự giảm dần và chỉ giữ điểm cao nhất của mỗi tài khoản cho tới khi đủ 10 dòng. Bản ghi cũ mang tên Player-xxxxxx được truy ngược chủ sở hữu qua bảng match_result: chỉ khi tìm được đúng một tài khoản khớp mới gộp vào tài khoản đó, ngược lại hiển thị là "Người chơi cũ". Việc gộp chỉ diễn ra trên bản sao dùng để hiển thị, dữ liệu lịch sử không bị ghi đè.

4.2.5. Bảo mật

Đăng nhập bằng form tại /login, mật khẩu băm bằng BCryptPasswordEncoder; tài khoản admin mặc định được tạo ở lần khởi động đầu tiên.

Tài nguyên tĩnh, trang chủ, đăng ký là công khai; /admin/** yêu cầu ROLE_ADMIN; mọi đường dẫn khác yêu cầu đăng nhập.

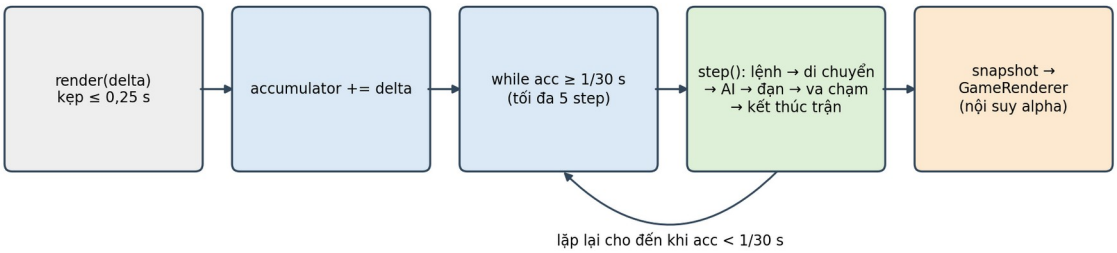
frameOptions.sameOrigin() cho phép nhúng game bằng iframe cùng nguồn nhưng chặn trang khác nhúng website.

CSRF được bỏ qua có chủ đích cho /api/game/**; các form Thymeleaf vẫn được bảo vệ.

4.3. Cài đặt phía client

4.3.1. Vòng lặp mô phỏng bước cố định

GameSimulation.advance nhận thời gian thực của mỗi khung hình, kẹp tối đa 0,25 giây để tránh "chạy bù" khi tab bị treo, rồi chạy tối đa 5 bước cố định 1/30 giây. Phần dư được dùng làm hệ số nội suy cho renderer (Hình 4.1).



Hình 4.1. Vòng lặp mô phỏng bước cố định

Trích GameSimulation.advance

```
public void advance(float rawDeltaTime) {
    double dt = Math.min(rawDeltaTime, GameplayConfig.MAX_FRAME_DELTA);
    accumulator += dt;
    int steps = 0;
    while (accumulator >= GameplayConfig.STEP_SECONDS
            && steps < GameplayConfig.MAX_STEPS_PER_FRAME) {
        step(GameplayConfig.STEP_SECONDS);
        accumulator -= GameplayConfig.STEP_SECONDS;
        steps++;
    }
    if (accumulator >= GameplayConfig.STEP_SECONDS) accumulator = 0;
}
```

Mỗi bước step() lần lượt: lưu vị trí cũ để nội suy, xử lý hàng đợi lệnh người chơi, cập nhật khiên và di chuyển xe người chơi, cập nhật nhiệm vụ và sinh xe theo đợt, cập nhật AI, cập nhật đạn, kiểm tra va chạm, dọn xe bị phá huỷ và kiểm tra điều kiện kết thúc trận.

4.3.2. Di chuyển theo ô và va chạm giữa các xe

Xe người chơi đi theo từng ô: mỗi lần bấm đi một ô, giữ phím thì tiếp tục sang ô kế tiếp; việc đổi hướng chỉ diễn ra ở tâm ô. Trước khi bắt đầu một bước, xe kiểm tra ô đích có bị tường chặn và có xe khác chiếm không. Một xe được coi là chiếm ô nếu hộp bao của nó giao với ô đó, hoặc nó đang di chuyển và đã giữ chỗ ô đó làm đích. AI dùng cùng quy tắc giữ chỗ nên hai xe không thể cùng lao vào một ô trong cùng một bước.

Trích PlayerTank.occupies

```
public boolean occupies(int col, int row, int tileSize) {
    if (!isAlive() || tileSize <= 0) return false;
    Rectangle cell = new Rectangle(col * tileSize + 1, row * tileSize + 1,
            tileSize - 2, tileSize - 2);
    if (getBoundingBox().intersects(cell)) return true;
    return movingDirection != null
            && col == (int) Math.floor(tileTargetX / tileSize)
            && row == (int) Math.floor(tileTargetY / tileSize);
}
```

Va chạm với tường dùng hộp bao thẳng trục (AABB) thu nhỏ 4 pixel mỗi cạnh (hitbox 32 × 32 trong ô 40 pixel), có trượt theo trục X/Y và đẩy nhẹ tối đa 6 pixel ở góc tường để xe không bị kẹt. Va chạm giữa hai viên đạn dùng kiểm tra liên tục (swept AABB) trên đoạn di chuyển trong một bước, để hai viên đạn bay ngược chiều nhanh không "xuyên qua" nhau giữa hai bước mô phỏng.

4.3.3. Trí tuệ nhân tạo của xe tăng

BuildsBotBrain điều khiển xe AI theo khung 60 Hz (hai khung cho mỗi bước mô phỏng 1/30 giây) với ba trạng thái SEARCHING, FORCE_ROTATION và ATTACK:

Quan sát: quét bốn tia thẳng – 10 ô phía trước và 6 ô các hướng khác – và bị tường, bụi che. Thấy đối thủ thì quay về hướng đó, bắn phát đầu sau 20 khung, sau đó mỗi 60 khung.

Tìm đường: khi đối thủ ở trong vùng hình thoi bán kính 5 ô, AI tìm đường bằng thuật toán Dijkstra trên lưới, trong đó ô gạch có chi phí cao hơn; gặp gạch trên đường thì bắn phá. Đường đi được tính lại sau mỗi 120 khung.

Tuần tra: không thấy ai thì đi thẳng, bị chặn thì rẽ trái/phải ngẫu nhiên, hết đường thì quay đầu.

Phản xạ: bị bắn khi chưa giao chiến thì quay về phía phát đạn; xe đồng minh định kỳ tìm đường tới vị trí người chơi.

Bộ sinh số ngẫu nhiên của AI được khởi tạo từ một seed cố định theo trận, và id đạn được cấp tuần tự, giúp mô phỏng có thể tái lập trong kiểm thử.

4.3.4. Giao tiếp mạng từ trình duyệt

Mã Java của game không thể gọi trực tiếp API trình duyệt, nên HtmlLauncher dùng JSNI (JavaScript Native Interface của GWT) để gọi XMLHttpRequest bất đồng bộ. Kết quả được trả về mã Java qua callback, đảm bảo chỉ kích hoạt đúng một trong hai sự kiện onLevelLoaded hoặc onLoadError:

Trích HtmlLauncher.BootstrapMapSource

```
private native void fetchBootstrap(String url, LevelCallback callback) /*-{
    var xhr = new XMLHttpRequest();
    xhr.open("GET", url, true);
    xhr.onreadystatechange = $entry(function() {
        if (xhr.readyState !== 4) return;
        if (xhr.status === 200) {
            var resp = JSON.parse(xhr.responseText);
            callback.@...LevelCallback::onLevelLoadedWithShop(...)(resp.levelId,
                JSON.stringify(resp.map), resp.runId, resp.mapVersion, ...);
        } else {
            callback.@...LevelCallback::onLoadError(Ljava/lang/String;)("HTTP " +
xhr.status);
        }
    });
    xhr.send();
}-*/;
```

LocalCampaignSession gắn mỗi yêu cầu bootstrap với một token; callback của yêu cầu cũ về muộn (ví dụ người chơi vừa chuyển màn) sẽ bị bỏ qua, tránh ghi đè phiên hiện tại.

4.3.5. Cửa hàng, túi đồ và radar

Cửa hàng (Workshop) được vẽ bằng ShopOverlay dưới dạng hộp thoại modal đè lên trận đấu. ShopCatalog dựng một danh sách cuộn duy nhất theo thứ tự: Đạn cơ bản, Vũ khí đặc biệt, Xe đồng minh, Phòng thủ, Sửa chữa, Hỗ trợ, Nghiên cứu và cuối cùng là mục Tiếp tục. Trạng thái túi đồ và cây nghiên cứu nằm trong PlayerShopState và được đồng bộ về server qua POST /api/game/shop/sync. Khi cửa hàng mở, ClientInputProcessor chuyển toàn bộ phím cho overlay.

Trong trận, cửa hàng chỉ mở khi xe người chơi đang đứng trên ô hồi sinh màu xanh lá (ô SPAWN_PLAYER chứa tâm xe); ô hồi sinh của địch, vùng sáng quanh bệ hay điểm hồi sinh dự phòng đều không được tính. Sau khi thắng, cửa hàng mở từ màn hình báo cáo để người chơi chuẩn bị cho màn kế tiếp. Giáp mua giữa trận được áp ngay lên

xe đang chạy và ghi vào điểm lưu đầu màn, nên chơi lại màn không làm mất giáp đã mua.

Trích LocalCampaignSession.canOpenShop

```
public boolean canOpenShop() {
    if (phase != Phase.PLAYING || sim == null || briefingOpen) return false;
    // Cửa hàng giữa hai màn vẫn mở từ báo cáo chiến thắng.
    return sim.getWorld().getStatus() == GameWorld.Status.VICTORY
            || sim.isLocalPlayerAtSpawnBase();
}
```

Phím Q mở menu túi đồ ngay trong trận: W/S hoặc phím mũi tên để chọn, Enter/Space để dùng, Q hoặc Esc để đóng; khi menu mở, xe dừng lại để không trôi theo phím đang giữ. Chọn vật phẩm Radar sẽ mở bản đồ tổng quan RadarOverlay, vẽ toàn bộ địa hình, tường, ô hồi sinh, cờ và các xe kèm vòng màu theo phe. RadarState có đồng hồ hiển thị 60 Hz riêng và không làm tiến thời gian mô phỏng; các đối tượng nhấp nháy theo chu kỳ 50 khung hình, còn Esc trong radar mở menu tạm dừng.

4.3.6. Màn hình kết thúc trận và giao diện tiếng Việt

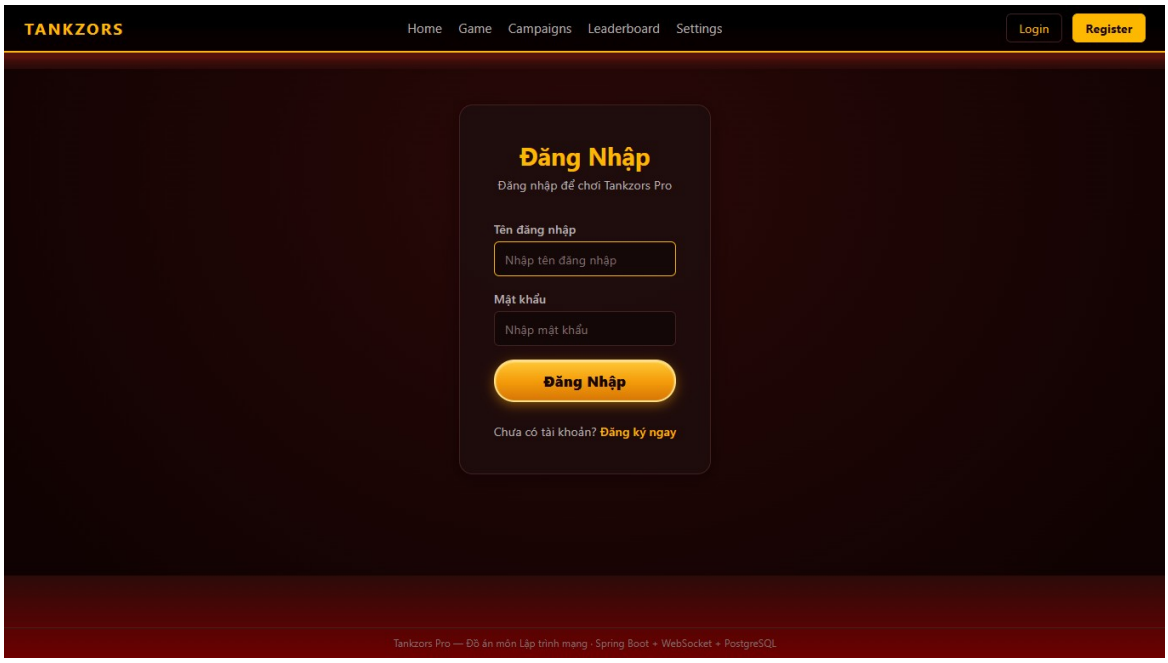
Khi thắng, VictoryOverlay hiển thị báo cáo nhiệm vụ lấy trực tiếp từ snapshot của trận vừa kết thúc (VictorySummary): Xu và Sao nhận được, Xu và Sao hiện có, thời gian nhiệm vụ, số xe địch đã tiêu diệt, máu còn lại và điểm. Màn hình chỉ có hai lựa chọn – tiếp tục qua cửa hàng sang màn kế, hoặc về danh sách màn. Khi thua, DefeatOverlay hỏi chơi lại (R) hay quay về danh sách màn (Esc). Cả hai màn hình đều điều khiển được bằng bàn phím lẫn chuột/chạm.

Phông BitmapFont mặc định của LibGDX không có glyph tiếng Việt, nên dự án sinh sẵn các phông bitmap vietnamese-font, game-font và shop-font (tệp .fnt và .png trong game-client/assets/fonts) bằng script game-client/tools/generate-vietnamese-font.ps1. Briefing, gợi ý trong bản đồ JSON, tên vật phẩm và nút bấm đều được chuyển sang tiếng Việt có dấu; ca kiểm thử VietnameseUiTextTest bảo đảm mọi ký tự xuất hiện trong nhiệm vụ và túi đồ đều có trong phông.

4.4. Giao diện chương trình

Các ảnh dưới đây được giữ từ tài liệu Tankzors do nhóm cung cấp, minh họa phiên bản sản phẩm trong báo cáo gốc. Ảnh không thay thế biên bản nghiệm thu hoặc một

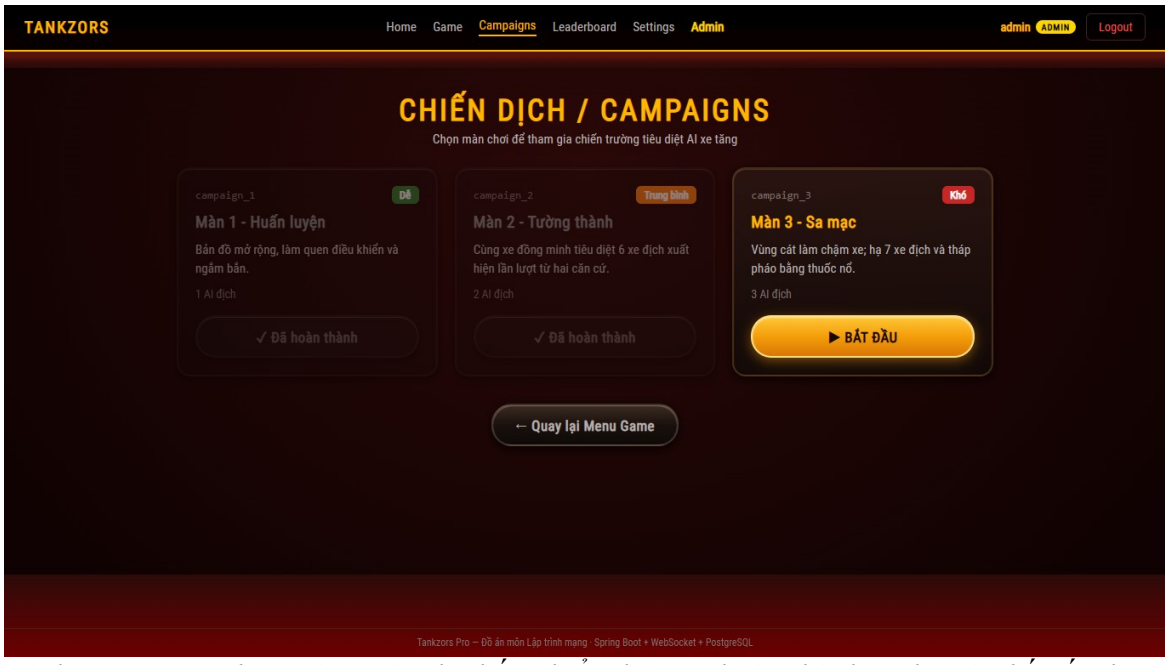
lần chạy lại trên mã nguồn đối chiếu; các khác biệt phiên bản được tổng hợp ở Phụ lục F.



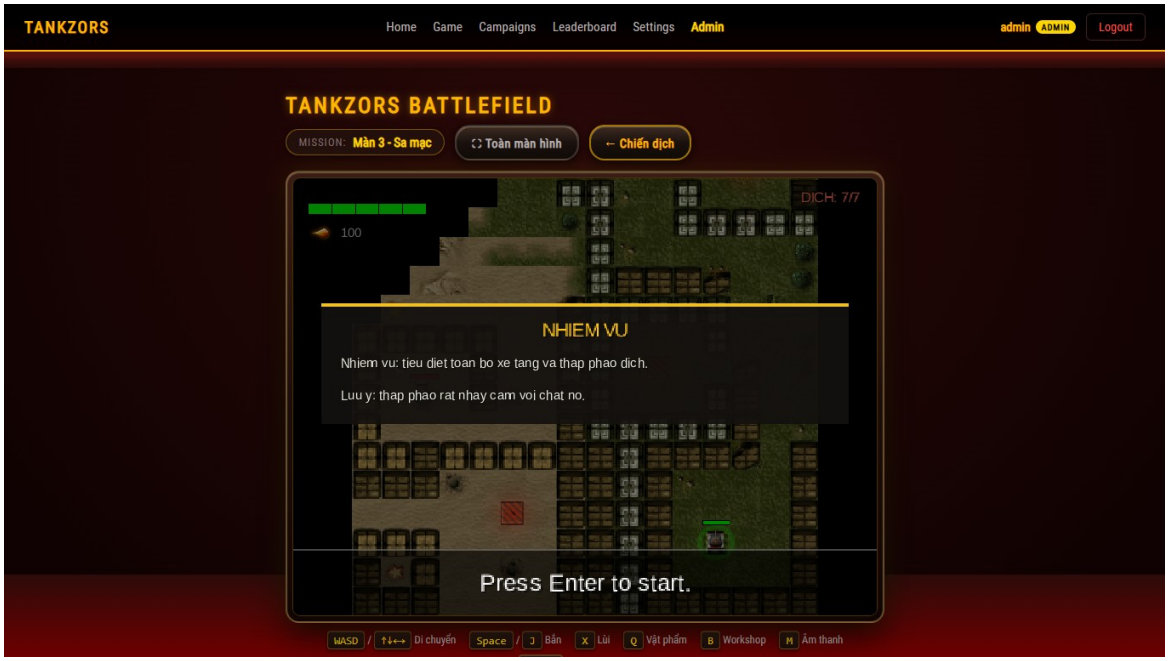
Hình 4.2. Trang đăng nhập



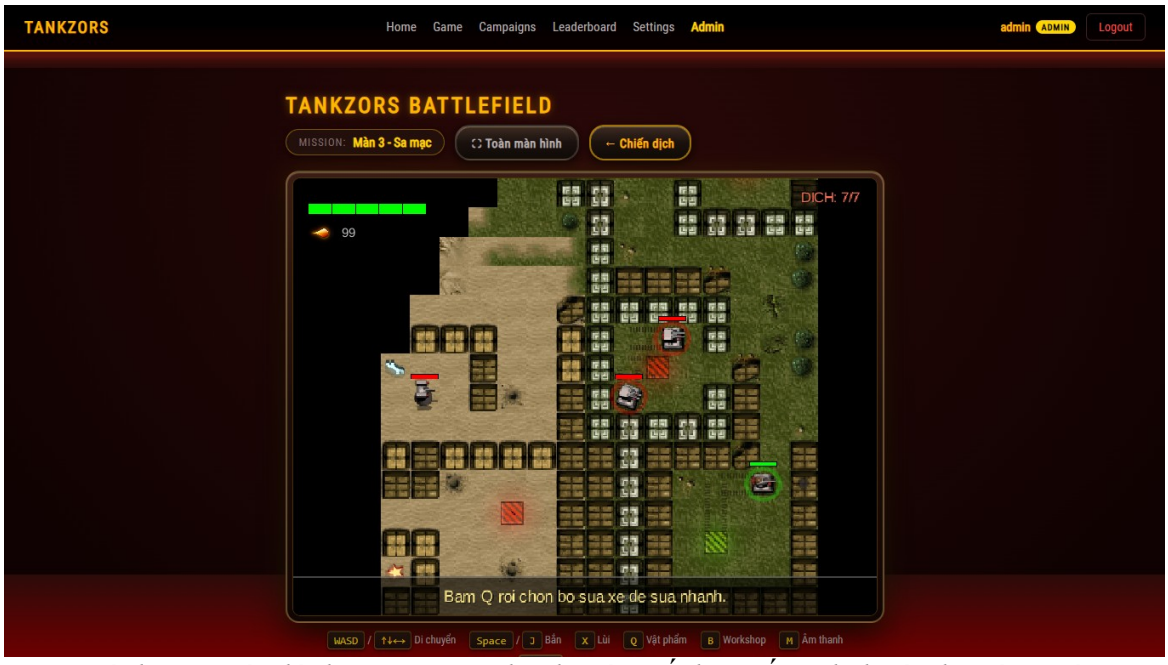
Hình 4.3. Trang chủ sau khi đăng nhập



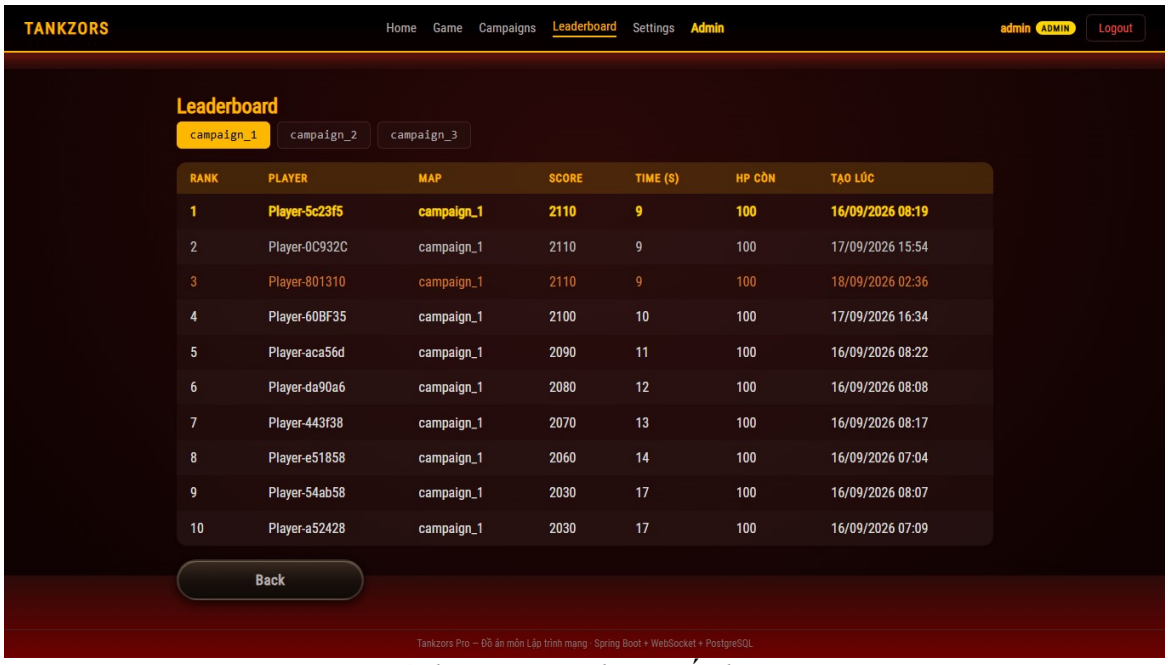
Hình 4.4. Trang chọn màn: màn đã thắng hiển thị "Đã hoàn thành", chỉ màn kế tiếp được bắt đầu



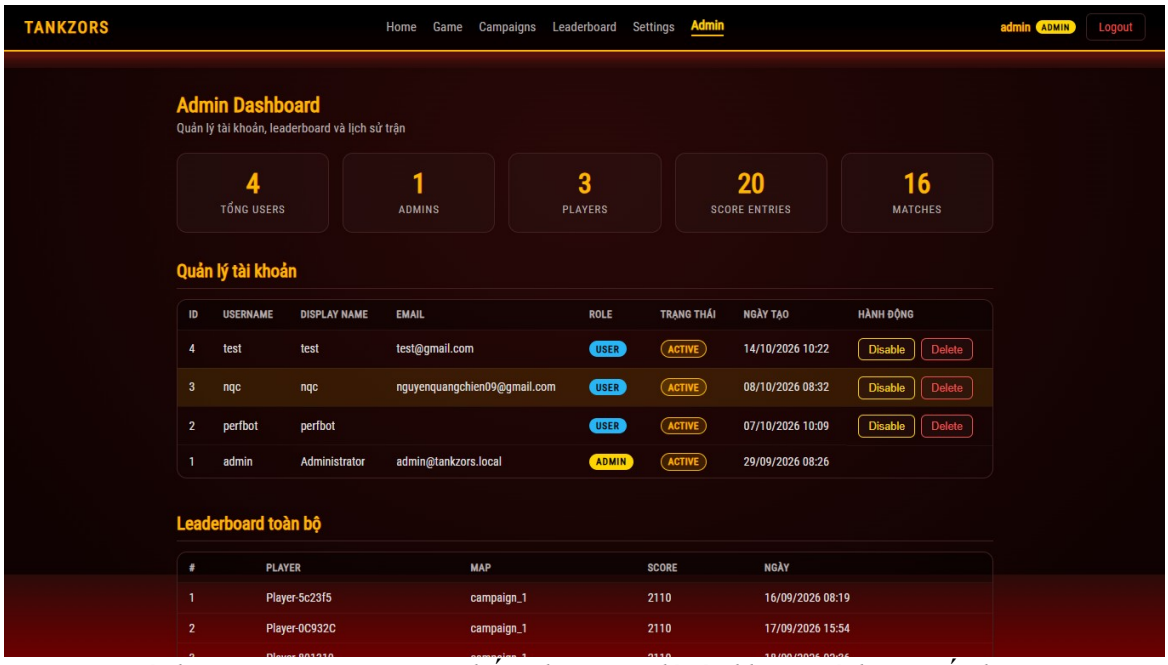
Hình 4.5. Màn hình nhiệm vụ trước khi vào trận



Hình 4.6. Màn hình trong trận: thanh máu, số đạn, số xe địch còn lại và gợi ý



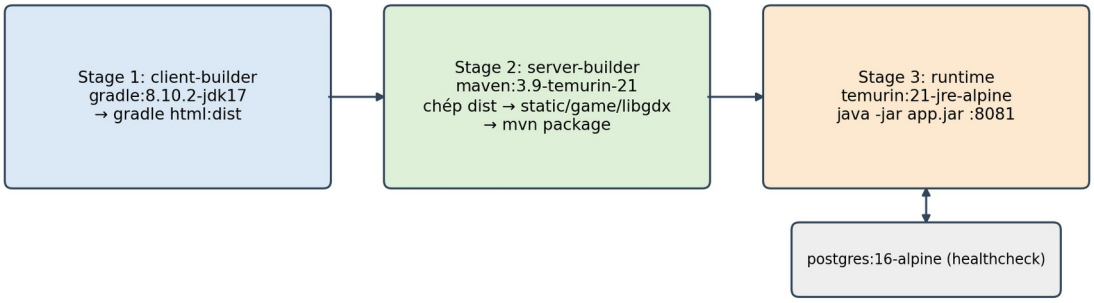
Hình 4.7. Trang bảng xếp hạng



Hình 4.8. Trang quản trị: thống kê, quản lý tài khoản và bảng xếp hạng

4.5. Đóng gói và triển khai

server-spring/Dockerfile gồm ba giai đoạn (Hình 4.9): giai đoạn 1 dùng Gradle + JDK 17 chạy html:dist để biên dịch client sang JavaScript; giai đoạn 2 dùng Maven + JDK 21, chép bản build client vào static/game/libgdx rồi đóng gói app.jar; giai đoạn 3 chỉ giữ JRE 21 trên Alpine để chạy ứng dụng. Thư mục bundle cũ luôn bị xoá trước khi chép nên JavaScript trong image luôn khớp với mã Java vừa biên dịch.



Hình 4.9. Quy trình build Docker nhiều giai đoạn

infra/docker-compose.yml chạy hai container: tankzors-postgres (PostgreSQL 16 có healthcheck pg_isready, dữ liệu lưu ở volume tankzors_pg_data) và tankzors-app (chỉ khởi động khi cơ sở dữ liệu đã sẵn sàng). Toàn bộ hệ thống được dựng bằng một lệnh:

```
docker compose -f infra/docker-compose.yml up --build -d
```

Sau khi container khởi động, Flyway tự áp dụng các migration và người dùng truy cập http://localhost:8081 để sử dụng.

Sau khi sửa mã nguồn hoặc asset, chạy lại lệnh up --build -d để tạo image ứng dụng mới. Cấu hình Compose đang đối chiếu không khai báo develop.watch; chế độ tự build bằng --watch chỉ phù hợp khi bổ sung các quy tắc theo dõi cần thiết.

```
docker compose -f infra/docker-compose.yml up --build -d
```

4.6. Chiến lược kiểm thử

Kiểm thử đơn vị gameplay sử dụng JUnit 5 tại game-client/core/src/test. Mô phỏng tách khỏi đồ hoạ cho phép kiểm tra di chuyển, va chạm, AI, điểm, phần thưởng, cửa hàng, radar và vòng đời session. Các ca gắn với thay đổi giao diện hoặc luật mới cần chạy trên đúng phiên bản mã nguồn.

Kiểm thử tích hợp server (Spring Boot Test, MockMvc, H2 ở chế độ tương thích PostgreSQL): chạy toàn bộ migration Flyway và kiểm tra controller, service, repository.

Kiểm thử tích hợp toàn hệ thống bằng Docker build, đảm bảo client biên dịch được bằng GWT và được đóng gói đúng vào server.

Thứ tự xác nhận đề xuất là chạy test gameplay, chạy test server với H2/MockMvc, build Docker rồi kiểm tra luồng đăng nhập – tải màn – chơi – gửi kết quả – mở màn kế – đọc lại sau khi khởi động lại. Docker build và test tự động là hai loại bằng chứng khác nhau; cần lưu log gắn với phiên bản mã nguồn.

4.7. Các ca kiểm thử tiêu biểu

Bảng 4.1. Một số ca kiểm thử tiêu biểu

STT	Ca kiểm thử	Kết quả mong đợi	Theo bản gốc
1	Xe người chơi đi vào ô có xe khác	Xe đứng yên ở tâm ô hiện tại	Đạt*
2	Xe đang di chuyển giữ chỗ ô đích	Ô đang rời và ô đích đều bị chiếm	Đạt*
3	Hai viên đạn bay ngược chiều	Va chạm liên tục, hai viên triệt tiêu	Đạt*

STT	Ca kiểm thử	Kết quả mong đợi	Theo bản gốc
4	Gửi lại kết quả cùng runId, cùng nội dung	200, alreadySaved = true, không cộng thưởng lần hai	Đạt*
5	Gửi kết quả cùng runId, khác nội dung	409 Conflict	Đạt*
6	Client khai thưởng vượt mức bản đồ	Server kẹp về mức tối đa của bản đồ	Đạt*
7	Thắng màn 1	Màn 1 bị khoá, màn 2 được mở	Đạt*
8	Bootstrap màn đã thắng	403 Forbidden	Đạt*
9	Thắng cả 3 màn	Không còn màn nào để chơi, không tự mở lại màn 1	Đạt*
10	Nhấn R sau khi thắng	Không tải lại màn đã thắng	Đạt*
11	Callback bootstrap cũ về muộn	Bị bỏ qua, không ghi đè phiên hiện tại	Đạt*
12	Trang chọn màn của tài khoản đã đăng nhập	Đọc đúng tiến trình theo user:<tên>	Đạt*
13	Xe bị phá huỷ khi vẫn còn mạng	Mất 1 mạng, hồi sinh tại ô xuất phát, giữ nguyên túi đồ	Đạt*
14	Hết 4 mạng nhưng còn vật phẩm Mạng bổ sung	Tự dùng Mạng bổ sung, trận tiếp tục	Đạt*
15	Bấm B khi không đứng trên ô hồi sinh xanh lá	Cửa hàng không mở	Đạt*
16	Bấm Tiếp tục khi tài khoản đang ở màn 2	Mở đúng màn 2; thắng hết thì về /campaigns kèm thông báo	Đạt*
17	Mở /play của màn đã thắng hoặc chưa mở	Chuyển về /campaigns kèm thông báo lỗi	Đạt*
18	Bảng xếp hạng có nhiều lượt của cùng tài khoản	Mỗi tài khoản chỉ hiện điểm cao nhất, tên là tên đăng nhập	Đạt*

** Cột kết quả được giữ theo tài liệu gốc, không phải kết quả chạy lại ngày 28/10/2026. Các ca liên quan bốn mạng, vị trí mở cửa hàng, nút Tiếp tục và tên tài khoản có khác biệt phiên bản được nêu tại Phụ lục F.*

4.8. Kết quả kiểm thử và phạm vi xác nhận

Bảng 4.2. Tổng hợp kết quả kiểm thử tự động

Phạm vi	Số lớp test	Số ca	Đạt	Lỗi	Bỏ qua
Client (game-client/core)	37	191	190	1	0
Server (server-spring)	15	76	74	1	1
Docker build tích hợp	–	1	1	0	0

Tài liệu gốc ghi nhận mốc 24/10/2026 tại commit 543ce0f: 264/267 ca tự động đạt, 2 ca lỗi và 1 ca bỏ qua; một lần Docker build tích hợp được báo cáo thành công riêng. Tài liệu mô tả hai lỗi ở SpecialAmmoShopTest (khác chuỗi giao diện có dấu/không dấu) và PageControllerTest.getLeaderboardRendersPageAndMapTabs (dữ liệu H2 còn lại giữa các ca). Đây là giải thích từ báo cáo gốc, chưa được xác nhận lại bằng log hoặc lần chạy mới trong quá trình biên tập. Vì vậy không suy rộng kết quả lịch sử thành kết quả kiểm thử của commit đang đối chiếu e4e9599.

Tổng số ca tự động được tính từ 191 ca client và 76 ca server, bằng 267 ca. Trong số đó có 190 + 74 = 264 ca đạt, 2 ca lỗi và 1 ca bỏ qua. Dòng Docker trong bảng là một lần kiểm tra tích hợp riêng, không cộng vào mẫu số 267. Tỷ lệ đạt tự động theo số liệu được cung cấp là khoảng 98,88%; tỷ lệ này không đo mức bao phủ mã hoặc hiệu năng.

V. KẾT QUẢ, THẢO LUẬN VÀ KẾT LUẬN

5.1. Kết quả đạt được

Hoàn thành trò chơi chơi được trên trình duyệt với 3 màn, AI địch và đồng minh, hệ thống vật phẩm và nghiên cứu, radar bản đồ, nhiệm vụ, mạng sống và chấm điểm.
Báo cáo gốc minh họa giao diện tiếng Việt, cửa hàng dạng modal và màn hình kết quả trận. Mức độ Việt hoá, các lớp overlay và bộ phông có khác biệt so với kho mã đối chiếu; cần kiểm tra theo phiên bản trước khi nghiệm thu.
Kiến trúc mạng rõ ràng: client chạy toàn bộ mô phỏng, server chỉ cung cấp REST API, lưu trữ và xác thực; server không chạy mô phỏng thứ hai.
Kết quả trận được lưu đúng một lần nhờ khoá idempotency runId và ràng buộc UNIQUE ở cơ sở dữ liệu.
Có xác thực, phân quyền, trang quản trị và bảng xếp hạng bền vững.
Hệ thống có workflow đóng gói Docker Compose và bộ kiểm thử đơn vị/tích hợp. Mốc kiểm thử lịch sử được trình bày tại mục 4.8; cấu hình build hiện tại cần chạy test riêng và không có sẵn quy tắc Docker Watch.

5.2. Hạn chế và giá trị của bằng chứng

Campaign là client-authoritative nên client bị chỉnh sửa vẫn có thể gửi kết quả giả trong các giới hạn cho phép. Server hiện chỉ kiểm tra một số giới hạn dữ liệu và kẹp phần thưởng, không tái lập diễn biến trận. Idempotency giải quyết yêu cầu lặp, không đồng nghĩa với chống gian lận hoàn chỉnh.
Dockerfile đóng gói server bằng -DskipTests. Vì vậy build thành công không chứng minh các test đạt; cần thực thi test riêng, kiểm tra nguồn log và bảo đảm các kết quả được gắn với đúng commit. Báo cáo gốc còn ghi nhận hai ca lỗi tại mốc lịch sử đã nêu.
Chưa có số liệu đo độ trễ, FPS theo cấu hình thiết bị, thử tải nhiều người dùng hoặc khảo sát trải nghiệm độc lập. Không thể kết luận khả năng mở rộng hay chất lượng trải nghiệm chỉ từ số lượng test và ảnh chụp. Nền tảng co-op chưa được coi là một luồng hoàn chỉnh từ lobby tới gameplay; một số mô tả trong bản báo cáo thuộc phiên bản khác với kho mã đối chiếu.

5.3. Thảo luận theo câu hỏi định hướng

5.3.1. Ranh giới mô phỏng và dịch vụ mạng

Câu hỏi thứ nhất được giải quyết bằng việc đặt gameplay tại game-client/core, còn Spring chỉ phục vụ web, REST, persistence, identity, room và relay. Renderer đọc snapshot và input phát lệnh qua session. Thiết kế này tránh duy trì hai mô phỏng độc lập, đồng thời cho phép tái sử dụng gameplay giữa desktop, HTML5 và host co-op. Giới hạn là kết quả campaign vẫn phụ thuộc vào client.

5.3.2. Lưu kết quả và tính bền vững

Câu hỏi thứ hai được giải quyết bằng transaction, ràng buộc UNIQUE(runId) và kiểm tra payload_hash. Gửi lại cùng nội dung có thể đọc kết quả đã lưu; gửi cùng runId nhưng thay nội dung trả xung đột. PostgreSQL và Flyway đảm nhiệm lưu trữ và quản lý schema. Cần tiếp tục kiểm tra yêu cầu đồng thời, sự ràng buộc runId với lượt đã cấp và định danh bảng xếp hạng; không suy diễn idempotency thành xác minh tính trung thực của người chơi.

5.3.3. Đóng gói và khả năng kiểm chứng

Câu hỏi thứ ba được giải quyết bằng Docker multi-stage: build HTML5 với JDK 17, build Spring với JDK 21, chạy bằng JRE 21. Bundle được tạo trong build rồi đóng gói vào app.jar; /game/libgdx/** là tài nguyên tĩnh. Để kiểm chứng đầy đủ cần tách bước test khỏi bước đóng gói, lưu log cùng commit và chạy lại luồng sử dụng trên image vừa tạo. Các bằng chứng lịch sử trong báo cáo là đầu vào tham khảo, chưa thay thế bước nghiệm thu này.

5.4. Kết luận

Đề tài đã xây dựng thành công Tankzors – một trò chơi xe tăng PvE hoàn chỉnh chạy trên trình duyệt, kết hợp website Spring Boot, client LibGDX HTML5 và cơ sở dữ liệu PostgreSQL. Qua quá trình thực hiện, nhóm đã vận dụng các kiến thức trọng tâm của học phần Lập trình mạng: mô hình client – server, giao thức HTTP và thiết kế REST API, quản lý phiên và xác thực, xử lý yêu cầu lặp lại bằng khoá idempotency,

cũng như cân nhắc lựa chọn mô hình quyền trạng thái phù hợp cho trò chơi một người chơi.

Bên cạnh đó, nhóm có thêm kinh nghiệm về tổ chức mã nguồn theo ranh giới trách nhiệm, viết kiểm thử tự động, quản lý schema cơ sở dữ liệu bằng migration và đóng gói ứng dụng bằng Docker.

5.5. Hướng phát triển

Tăng khả năng xác minh: ghi lại input và seed của trận, ràng buộc runId với danh tính và màn đã cấp, tăng kiểm tra dữ liệu kết quả và giao dịch cửa hàng. Nếu nghiên cứu kiểm chứng bằng replay, cần tái sử dụng cùng nguồn gameplay hoặc một cơ chế xác minh riêng đã được thiết kế rõ; không tạo một gameplay engine độc lập trong Spring.

Bổ sung thêm màn chơi và trình chỉnh sửa bản đồ.

Thiết lập CI tự động chạy kiểm thử và build Docker cho mỗi lần commit.

Hoàn thiện luồng co-op từ lobby tới game frame và bổ sung kiểm thử protocol v2, quyền host, input trùng/cũ, host mất kết nối. Tiếp tục chuẩn hoá giao diện tiếng Việt, đồng nhất tên tài khoản trên bảng xếp hạng và cập nhật các test theo đúng phiên bản sản phẩm. Đo hiệu năng trình duyệt, thời gian tải và độ trễ thực tế trước khi mở rộng phạm vi triển khai.

TÀI LIỆU THAM KHẢO

Docker Inc. (n.d.). Multi-stage builds. Docker Documentation. https://docs.docker.com/build/building/multi-stage/

Docker Inc. (n.d.). Docker Compose. Docker Documentation. https://docs.docker.com/compose/

Fiedler, G. (2004). Fix your timestep! https://gafferongames.com/post/fix_your_timestep/

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc9110

Gambetta, G. (n.d.). Fast-paced multiplayer. https://www.gabrielgambetta.com/client-server-game-architecture.html

Kurose, J. F., & Ross, K. W. (2021). Computer networking: A top-down approach (8th ed.). Pearson.

libGDX. (n.d.). HTML5 backend and GWT specifics. libGDX Wiki. https://libgdx.com/wiki/html5-backend-and-gwt-specifics

Redgate. (n.d.). Flyway documentation. https://documentation.red-gate.com/fd

Spring. (n.d.). Spring Boot reference documentation (3.3). https://docs.spring.io/spring-boot/3.3/reference/index.html

Spring. (n.d.). Spring Security reference. https://docs.spring.io/spring-security/reference/

The PostgreSQL Global Development Group. (n.d.). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/

Nhóm phát triển Tankzors. (2026). Mã nguồn và tài liệu dự án: README.md, docs/architecture.md, docs/protocol-v2.md, server-spring/Dockerfile và infra/docker-compose.yml. Bản kho mã đối chiếu: e4e9599.

Nhóm sinh viên Vũ Anh Tuấn, Nguyễn Quang Chiến và Trần Quang Lâm. (2026). Báo cáo bài tập lớn: Xây dựng game xe tăng Tankzors [Tài liệu đầu vào: BaoCao_Tankzors_moi.docx].

PHỤ LỤC

Phụ lục A. Cài đặt và sử dụng hệ thống

A.1. Yêu cầu môi trường

Máy cài Docker Desktop và có kết nối Internet ở lần build đầu. Không cần cài Java, Gradle, Maven hay PostgreSQL.

A.2. Khởi động

```
docker compose -f infra/docker-compose.yml up --build -d
docker compose -f infra/docker-compose.yml ps        # kiểm tra 2 container đang chạy
docker compose -f infra/docker-compose.yml logs -f app
```

Khi log xuất hiện dòng Started TankzorsApplication, truy cập http://localhost:8081. Tài khoản quản trị mặc định của môi trường phát triển là admin / admin123.

Tài khoản quản trị mặc định chỉ dành cho môi trường phát triển. Trước khi triển khai sử dụng thực tế cần đổi thông tin mặc định và thiết lập cấu hình bảo mật phù hợp.

A.3. Cập nhật, dừng và khởi động lại

```
docker compose -f infra/docker-compose.yml up --build -d
docker compose -f infra/docker-compose.yml down
docker compose -f infra/docker-compose.yml up -d
```

Dùng up --build -d sau khi sửa mã; dùng up -d khi chỉ bật lại image đã có. Lệnh down dừng container và giữ volume. Không thêm -v nếu cần giữ tài khoản, kết quả và tiến trình. Khi trình duyệt dùng bundle cũ, tải lại bằng Ctrl + Shift + R.

Phụ lục B. Thực thi và lưu bằng chứng kiểm thử

```
mvn -f server-spring/pom.xml test                # server: JDK 21 + Maven
cd game-client && ./gradlew :core:test           # client: JDK 17
```

Hai lệnh trên cần JDK tương ứng trên môi trường thực thi: JDK 21 cho Maven server và JDK 17 cho Gradle client. Workflow chuẩn của dự án cô lập toolchain bằng container; có thể chạy test trong container build phù hợp. Dockerfile phát hành hiện không thay cho một bước test riêng.

Bằng chứng	**Vị trí hoặc nội dung cần lưu**
Client test	game-client/core/build/test-results/test và reports/tests/test
Server test	server-spring/target/surefire-reports
Build tích hợp	Log docker compose build; commit; thời điểm; image ID
Kiểm tra sử dụng	Đăng nhập; tải màn; lưu kết quả; gửi lại cùng runId; tiến trình sau khởi động lại
Môi trường	Phiên bản Docker; cấu hình trình duyệt; thông số thiết bị nếu có đo FPS/độ trễ

Nếu có ca thất bại, lưu lỗi quan sát được, bước tái hiện và kết quả mong đợi. Kiểm tra giả thuyết bằng một lần chạy lại trước khi kết luận nguyên nhân. Không thay ô lỗi bằng "Đạt" khi chưa có bằng chứng xác nhận.

Phụ lục C. Phím điều khiển

Bảng C.1. Phím điều khiển trong trò chơi

Phím	**Chức năng**
W A S D / phím mũi tên	Di chuyển theo từng ô; chọn mục trong menu
X	Lùi (không xoay đầu xe)
Space / J	Bắn; giữ phím để bắn liên tục
Q	Mở/đóng menu túi đồ để chọn đạn, vật phẩm, Radar
1 – 6	Chọn nhanh loại đạn
B	Mở cửa hàng khi đứng trên ô hồi sinh xanh lá (hoặc sau khi thắng)
M	Bật/tắt âm thanh
Enter / Space	Đóng briefing để bắt đầu màn; xác nhận lựa chọn trong menu và màn hình kết quả
Tab / ← →	Đổi lựa chọn trên màn hình thắng/thua
R	Chơi lại màn khi thua
Esc	Quay về trang chọn màn; trong radar: mở menu tạm dừng

Bảng giữ cách điều khiển theo phiên bản báo cáo gốc. Giới hạn vị trí mở cửa hàng và một số thao tác ở màn hình kết quả cần đối chiếu với bản chạy cụ thể; xem Phụ lục F.

Phụ lục D. Tóm tắt giao tiếp campaign và co-op

Luồng	Thông điệp / đường dẫn	Vai trò
Tải campaign	GET /api/game/bootstrap	Cấp map, runId và metadata; kiểm tra màn được mở
Lưu kết quả	POST /api/game/results	Lưu idempotent; trả 200 hoặc 409 theo runId/payload
Cửa hàng	POST /api/game/shop/sync	Đồng bộ trạng thái cửa hàng và túi đồ
Tài nguyên HTML5	/game/libgdx/**	Tệp tĩnh được Spring phục vụ; không phải WebSocket
Phòng co-op	/coop; JSON envelope v2	CREATE_ROOM, JOIN_ROOM, READY, START_MATCH, LEAVE_ROOM
Guest → host	PLAYER_INPUT	Relay chuyển input; host xử lý gameplay
Host → guest	WORLD_STATE, MATCH_RESULT	Relay chuyển snapshot/kết quả; guest hiển thị

Ví dụ envelope minh họa cấu trúc, không phải log đo thực tế:

```
{
  "version": "v2", "type": "PLAYER_INPUT",
  "roomId": "room-example", "runId": "run-example", "seq": 1,
  "payload": {"direction": "UP", "moving": true, "shoot": false}
}
```

senderId được relay gắn từ session handshake. Chỉ host có quyền phát WORLD_STATE/MATCH_RESULT; server không chạy simulation. Việc hoàn thiện launcher, xử lý kết nối và kiểm thử xuyên suốt là điều kiện để nghiệm thu co-op.

Phụ lục E. Cấu trúc mẫu và đối chiếu từng mục

Tài liệu mẫu là báo cáo nghiên cứu về môi trường hội thoại tiếng Anh. Báo cáo Tankzors tiếp thu trình tự trình bày và hệ định dạng, còn đối tượng, phương pháp và bằng chứng được chuyển sang đồ án phần mềm. Không chuyển khảo sát, người tham gia hoặc kết quả nghiên cứu của đề tài mẫu thành dữ liệu của Tankzors.

Bảng E.1. Đối chiếu cấu trúc tài liệu mẫu với báo cáo Tankzors

Mục trong tài liệu mẫu	**Mục tương ứng và nội dung của Tankzors**
Bìa ngoài và bìa trong	Tên Học viện/Khoa, học phần, tên đề tài Việt/Anh, giảng viên, ba sinh viên và mã sinh viên, địa điểm/thời gian
Student Declaration	Lời cam đoan: trách nhiệm nội dung, nguồn tài liệu và phạm vi số liệu
Acknowledgment	Lời cảm ơn giảng viên, cơ sở đào tạo và người góp ý
Table of Contents	Mục lục; bổ sung danh mục hình, bảng, thuật ngữ để tra cứu báo cáo kỹ thuật
I. Introduction	I. Giới thiệu: lý do, bài toán, mục tiêu, phạm vi, yêu cầu, công nghệ, phương pháp và bố cục
II. Literature Review	II. Cơ sở lý thuyết: client–server, TCP/IP, HTTP/REST, authority, fixed-step, xác thực và công nghệ
III. Methodology – Research Design	III. Phương pháp, phân tích và thiết kế: quy trình phát triển, ca sử dụng, kiến trúc, DB, API, gameplay
Participants and Sampling	Chuyển thành tác nhân/phạm vi sử dụng; không lập mẫu khảo sát khi không có nghiên cứu người dùng
Data Collection Instruments	Chuyển thành công cụ kiểm thử và bằng chứng: JUnit, MockMvc, H2, Docker, log và ảnh minh họa
Data Collection Procedures	Chuyển thành luồng bootstrap–simulation–result và quy trình build/kiểm thử/nghiệm thu
IV. Piloting of Data-Collection Instruments	IV. Cài đặt và kiểm thử: client, server, giao diện, đóng gói, ca kiểm thử và số liệu lịch sử
V. Findings and Discussion	V. Kết quả, thảo luận và kết luận: thành quả, hạn chế, trả lời câu hỏi định hướng và hướng phát triển
References	Tài liệu tham khảo: tài liệu nền tảng, tài liệu công nghệ và tư liệu dự án
Appendices A–G	Phụ lục A–F: cài đặt; kiểm thử; phím; giao tiếp; đối chiếu cấu trúc; giới hạn phiên bản

Phụ lục F. Đối chiếu phiên bản và giới hạn xác nhận

Các điểm sau được ghi nhận khi đối chiếu tài liệu đầu vào với kho mã có HEAD e4e9599. Mục đích là làm rõ nguồn bằng chứng và tránh đồng nhất mô tả/ảnh của một phiên bản với trạng thái của phiên bản khác. Việc biên tập báo cáo không bao gồm sửa mã hoặc thực thi lại toàn bộ test/build của dự án.

Bảng F.1. Các khác biệt cần kiểm tra theo phiên bản

Hạng mục	**Theo tài liệu Tankzors đầu vào**	**Quan sát tại mã nguồn đối chiếu**
Mạng sống	Mỗi màn có 4 mạng, sau đó dùng Mạng bổ sung	GameSimulation xử lý hồi sinh theo mode cướp cờ hoặc vật phẩm ADD_LIFE; chưa xác nhận quy tắc 4 mạng chung cho mọi màn
Mở cửa hàng	Chỉ trên ô hồi sinh xanh lá hoặc sau chiến thắng	LocalCampaignSession.canOpenShop kiểm tra phase PLAYING và sim khác null; chưa có kiểm tra vị trí trong phương thức này
Tiếp tục campaign	getContinueLevel chọn màn xa nhất; /play kiểm tra quyền trước khi render	PlayerProgressService chưa có getContinueLevel; /play mặc định campaign_1 và kiểm tra map tồn tại; bootstrap vẫn kiểm tra quyền mở màn
Tên bảng xếp hạng	Dùng tên tài khoản và gộp dữ liệu tên cũ	ResultService.playerNameFor tạo Player- kèm 6 ký tự đầu playerId; topScores gọi truy vấn top 10, chưa có logic gộp theo tài khoản
Giao diện tiếng Việt	Có VictoryOverlay, DefeatOverlay, VietnameseUiTextTest và nhiều bitmap font	Chưa tìm thấy các lớp/test đó trong danh sách tệp hiện tại; assets/fonts có shop-font và RobotoCondensed_Regular.ttf
Docker Watch	Compose có develop.watch	infra/docker-compose.yml hiện chỉ khai báo services/volumes, chưa có develop.watch
Co-op	Bản báo cáo gốc tập trung một người chơi	Có /coop, RoomRegistry, HostGameSession, GuestGameSession; docs/architecture.md nêu việc nối lobby–launcher còn cần hoàn thiện
Số liệu kiểm thử	24/10/2026, commit 543ce0f, 264/267 đạt	Số liệu lịch sử do tài liệu cung cấp; chưa có log chạy lại tương ứng HEAD e4e9599 trong lần biên tập

Khi nghiệm thu, cần thống nhất báo cáo với phiên bản trình diễn và log kiểm thử. Ranh giới kiến trúc vẫn nhất quán: gameplay ở client core, campaign dùng REST, co-op mục tiêu dùng /coop v2 và Spring không mô phỏng.